



हेलो एंड नमस्ते। यह वो वीडि...
TXT 1.07MB

create notes for me out of this transcript

JavaScript Complete Notes (From Transcript)

1. प्रीरिक्विजिट्स (Prerequisites)

Words vs Keywords

- **Words:** कोई भी सामान्य शब्द जिसका कोई विशेष मतलब नहीं होता (जैसे: हर्ष, श्याम)
- **Keywords:** ऐसे शब्द जिनका JavaScript में विशेष मतलब होता है (जैसे: let, const, if, else, function, for, while)

JavaScript की History

- Brendan Eich ने Netscape में काम करते हुए सिर्फ 15 दिनों में बनाई
- पहले नाम था Mocha, बाद में JavaScript रखा गया (Java की popularity की वजह से)
- JavaScript और Java का कोई संबंध नहीं है
- ECMAScript (ES5, ES6) community द्वारा maintain किया जाता है
- ES5 (पुराना) → ES6 (नया, major update)

Compiler कैसे काम करता है?

- JavaScript code को line by line execute करती है
- पहले declaration होता है, फिर initialization

2. Variables और Declarations

Variable क्या है?

- Container जो data को store करता है
- Program की life cycle तक data याद रखता है

Variable बनाने के 3 तरीके:

Keyword	Scope	Re-declare	Re-assign	Hoisting
var	Function	✓ हाँ	✓ हाँ	undefined
let	Block	✗ नहीं	✓ हाँ	Reference Error
const	Block	✗ नहीं	✗ नहीं	Reference Error

Declaration vs Initialization

- **Declaration:** `let a;` (variable बनाया, value नहीं दी)
- **Initialization:** `let a = 12;` (पहली value दी)

Re-assignment vs Re-declaration

- **Re-assignment:** `a = 32;` (value बदलना) - सब में allowed
- **Re-declaration:** `let a = 12; let a = 13;` - सिर्फ **var** में allowed

Temporal Dead Zone (TDZ)

- `let` और `const` में declaration से initialization के बीच का area
- उस area में variable access नहीं कर सकते
- Error: Cannot access 'a' before initialization

Hoisting

- **var:** Hoist होता है, value undefined मिलती है
- **let/const:** Hoist होता है लेकिन value नहीं मिलती, **Reference Error** आता है

3. Scope

Scope Type	Meaning	Example
Global	पूरे code में accessible	Function/Block के बाहर
Function	सिर्फ function के अंदर	<code>var</code> का scope
Block	सिर्फ <code>{ }</code> के अंदर	<code>let</code> , <code>const</code> का scope

4. Data Types

Primitive Types (7)

1. **String** - `"Harsh"` , `'Hello'` , ``Backtick``
2. **Number** - `12` , `12.3`
3. **Boolean** - `true` , `false`
4. **Null** - जानबूझकर खाली value
5. **Undefined** - variable बनाया, value नहीं दी
6. **Symbol** - Unique value
7. **BigInt** - बहुत बड़े numbers (`123n`)

Reference Types (3)

1. **Array** - `[1, 2, 3]`
2. **Object** - `{name: "Harsh"}`

3. Function - `function() {}`

Primitive vs Reference

- **Primitive:** Direct copy होती है (real copy)
- **Reference:** Reference pass होता है (original change होता है)

Null vs Undefined

- **Null:** जानबूझकर value नहीं दी (manual)
- **Undefined:** value ही नहीं दी (JavaScript engine देता है)

Dynamic Typing

- JavaScript में variable का type बदल सकते हैं
- `let a = 12; a = true; a = "Harsh";` - सब allowed

Typeof Quirks

- `typeof null` → "object" (bug)
- `typeof []` → "object"
- `typeof NaN` → "number"
- `typeof function` → "function"

5. Operators

Arithmetic Operators

Operator	Meaning	Example
+	Addition / Concatenation	<code>2+3=5</code> , <code>"5"+1="51"</code>
-	Subtraction	<code>5-1=4</code>
*	Multiplication	<code>2*3=6</code>
/	Division	<code>12/6=2</code>
%	Modulus (Remainder)	<code>12%5=2</code>
**	Exponentiation	<code>2**3=8</code>

Comparison Operators

- `==` - Value check (type नहीं) - **Loose**
- `===` - Value + Type check - **Strict** ✓
- `!=` - Not equal (loose)
- `!==` - Not equal (strict) ✓

- > , < , >= , <=

Logical Operators

- && (AND) - दोनों true होने चाहिए
- || (OR) - कोई एक true होना चाहिए
- ! (NOT) - उल्टा कर देता है

Assignment Operators

- = , += , -= , *= , /= , %=

Unary Operators

- ++a (Pre-increment) - पहले बढ़ाओ, फिर print
- a++ (Post-increment) - पहले print, फिर बढ़ाओ
- --a , a-- (Decrement)
- + (String to Number conversion)
- - (Negation)
- ! (Truthy/Falsy check)
- typeof

Ternary Operator

```
js
condition ? trueValue : falseValue
```

Truthy vs Falsy Values

Falsy Values (8):

- false , 0 , -0 , 0n , "" , null , undefined , NaN

Truthy: बाकी सब कुछ

6. Control Flow

if / else if / else

```
js
if (condition) {
  // code
} else if (condition2) {
  // code
} else {
  // code
}
```

Switch Case

```
js

switch(value) {
  case 1:
    // code
    break;
  case 2:
    // code
    break;
  default:
    // code
}
```

- **Break** जरूरी है, वरना next case भी चलेगा (fall through)

Early Return Pattern

- बड़े if-else blocks की जगह छोटे-छोटे returns
- Code ज्यादा clean और readable होता है

7. Loops

For Loop

```
js

for (let i = 0; i < 10; i++) {
  // code
}
```

कब use करें: जब Start, End, और Change तीनों पता हों

While Loop

```
js

let i = 0;
while (i < 10) {
  // code
  i++;
}
```

कब use करें: जब सिर्फ रुकने की condition पता हो

Do-While Loop

```
js

do {
  // code
} while (condition);
```

- कम से कम **एक बार** जरूर चलेगा

Break और Continue

- **Break:** Loop को वहीं रोक दो
- **Continue:** उस iteration को skip करो, next पे जाओ

Loop Selection Guide

- **For:** Start, End, Change पता है
- **While:** सिर्फ रुकने की condition पता है
- **Do-While:** कम से कम एक बार चलाना है

8. Functions

Function के प्रकार

1. Function Declaration (Statement)

```
js

function greet() {
  console.log("Hello");
}
```

- **Hoisted** होता है (पहले call कर सकते हैं)

2. Function Expression

```
js

const greet = function() {
  console.log("Hello");
};
```

- **Hoisted नहीं** होता (पहले call नहीं कर सकते)

3. Arrow Function (Fat Arrow)

```
js

const greet = () => {
  console.log("Hello");
};
```

- **Hoisted नहीं** होता
- **this** keyword अपना नहीं होता

Parameters vs Arguments

- **Parameters:** Function declaration में लिखे variables
- **Arguments:** Function call में भेजी values

Default Parameters

js

```
function add(a = 0, b = 0) { return a + b; }
```

Rest Operator (...)

js

```
function sum(...numbers) {  
  // numbers एक array है  
}
```

Spread Operator (...)

- Array/Object को फैलाने के लिए

Return Values

- Function कुछ return कर सकता है
- Return नहीं किया तो `undefined` return होता है
- Return के बाद का code execute नहीं होता

First-Class Functions

- Functions को value की तरह treat किया जा सकता है
- Variable में store कर सकते हैं
- Function में pass कर सकते हैं

Higher-Order Functions

- Function जो function return करे या parameter में function ले

Pure vs Impure Functions

- **Pure:** बाहर की value नहीं बदलता
- **Impure:** बाहर की value बदल देता है (side effects)

Closures

- Function जो return करे एक और function
- Return वाला function parent function का variable use करे

Lexical Scoping

- Variable का scope उसकी physical position से तय होता है

IIFE (Immediately Invoked Function Expression)

js

```
(function() {  
  // code  
})();
```

Hoisting Differences

- **Function Declaration:** Hoisted
- **Function Expression:** Hoisted
- **Arrow Function:** Hoisted

9. Arrays

Array क्या है?

- एक से ज्यादा values को एक साथ store करने का container
- Square brackets [] से बनता है
- Index 0 से शुरू होता है

Array बनाने के तरीके

js

```
let arr = [1, 2, 3];  
let arr2 = new Array(1, 2, 3);
```

Array Methods

Method	क्या करता है
push()	Last में add
pop()	Last से remove
unshift()	Start में add
shift()	Start से remove
splice()	कहीं से भी remove/add (original change)
slice()	Copy निकालता है (original change नहीं)
reverse()	उल्टा करता है (original change)
sort()	Sort करता है (original change)
map()	नया array बनाता है
filter()	Condition से filter करता है

Method	क्या करता है
<code>reduce()</code>	Single value में reduce करता है
<code>find()</code>	पहला matching element
<code>some()</code>	कोई एक match करे तो true
<code>every()</code>	सब match करें तो true
<code>forEach()</code>	हर element पे function चलाए
<code>Object.keys()</code>	सारी keys का array
<code>Object.entries()</code>	Key-value pairs का array

Sort का Big Blunder

```
js

// गलत (numbers को string बना देता है)
arr.sort(); // [100, 20, 3] → [100, 20, 3]

// सही
arr.sort((a, b) => a - b); // Ascending
arr.sort((a, b) => b - a); // Descending
```

Destructuring

```
js

let [a, b] = [1, 2];
let [a, , c] = [1, 2, 3]; // बीच वाला skip
```

Spread Operator

```
js

let arr2 = [...arr1]; // Copy
let merged = [...arr1, ...arr2];
```

Map vs ForEach

- **Map:** नया array return करता है, return जरूरी
- **ForEach:** कुछ return नहीं करता, सिर्फ loop

10. Objects

Object क्या है?

- Key-Value pairs का collection

- Curly braces `{}` से बनता है

Object बनाना

```
js

let obj = {
  name: "Harsh",
  age: 26,
  email: "test@test.com"
};
```

Access करने के तरीके

```
js

obj.name;           // Dot notation
obj["name"];        // Bracket notation
```

Dot vs Bracket:

- Dot: Literally वही नाम ढूँढता है
- Bracket: Variable की value को convert करके ढूँढता है

Nested Objects & Deep Access

```
js

user.address.location.lng; // Deep access
```

Optional Chaining (`?.`)

```
js

user?.address?.city; // Error नहीं देगा, undefined देगा
```

Computed Properties

```
js

let key = "role";
let obj = { [key]: "admin" }; // { role: "admin" }
```

Looping on Objects

for...in (Keys के लिए):

```
js

for (let key in obj) {
  console.log(key, obj[key]);
}
```

Object.keys():

```
js

Object.keys(obj); // ["name", "age", "email"]
```

Object.entries():

```
js

Object.entries(obj); // [["name", "Harsh"], ["age", 26], ...]
```

Object Copying**Shallow Copy (Spread):**

```
js

let obj2 = { ...obj };
```

- Top level values copy होती हैं
- Nested objects reference pass होते हैं

Object.assign():

```
js

let obj2 = Object.assign({}, obj);
```

Deep Clone (JSON method):

```
js

let obj2 = JSON.parse(JSON.stringify(obj));
```

- Nested objects की real copy होती है

Destructuring Objects

```
js

let { name, age } = obj;
let { name: firstName } = obj; // नाम बदलना
let { address: { city } } = obj; // Nested
```

Shallow vs Deep Copy

- **Shallow:** Nested objects reference pass होते हैं
- **Deep:** Nested objects की real copy होती है

11. Important Concepts Summary

Hoisting Summary

Type	Hoisted?	Value
var	✓	undefined
let	✓	Reference Error (TDZ)
const	✓	Reference Error (TDZ)
Function Declaration	✓	Function
Function Expression	✗	-
Arrow Function	✗	-

Scope Summary

Keyword	Scope
var	Function
let	Block
const	Block

Copy Summary

Type	Copy Method
Primitive	Direct
Array (shallow)	Spread [...arr]
Array (deep)	JSON.parse(JSON.stringify(arr))
Object (shallow)	Spread {...obj}
Object (deep)	JSON.parse(JSON.stringify(obj))

12. Practice Questions Covered

Loops

- 1 से 10 तक print
- 10 से 1 तक print (reverse loop)
- Even numbers 1-20
- Odd numbers 1-15
- Multiplication table of 5
- Sum of numbers 1-100

7. Numbers divisible by 3 (1-50)
8. Even/Odd check
9. Count numbers divisible by 3 and 5
10. First multiple of 7 पे stop
11. Skip multiples of 3
12. First 5 odd numbers

Functions

- Grade calculator
- Rock Paper Scissors logic
- BMI calculator
- Discount calculator (HOF)
- Counter (Closure)

Arrays

- Create, access, modify
- Push, pop, shift, unshift, splice, slice
- Map, filter, reduce, find, some, every
- Destructuring, spread
- Merge, clone

Objects

- Create, access (dot/bracket)
- Nested objects
- Looping (for...in, Object.keys, Object.entries)
- Copy (shallow/deep)
- Optional chaining
- Computed properties

13. Common Confusions

1. **let vs var**: let block-scoped, var function-scoped
2. **== vs ===**: === type भी check करता है
3. **null vs undefined**: null manual, undefined automatic
4. **Primitive vs Reference**: Reference copy में original change होता है
5. **Map vs ForEach**: Map return करता है, ForEach नहीं
6. **Splice vs Slice**: Splice original change, Slice copy
7. **Function Declaration vs Expression**: Declaration hoisted, Expression नहीं
8. **Shallow vs Deep Copy**: Nested objects के लिए deep copy जरूरी
9. **Closure vs HOF**: Closure में parent का variable use होता है

10. **TDZ:** let/const में declaration से initialization तक access नहीं

14. Quick Reference Card

```
js

// Variables
let a = 12;           // Block scope, re-assign ✓
const b = 12;         // Block scope, re-assign ✗
var c = 12;           // Function scope, re-declare ✓

// Data Types
"String", 12, true, null, undefined, Symbol(), 123n
[], {}, function() {}

// Operators
+ - * / % **
== === != !== > < >= <=
&& || !
= += -= *= /= %=
++ -- + - ! typeof
condition ? true : false

// Control Flow
if/else if/else, switch/case, ternary

// Loops
for, while, do-while
break, continue

// Functions
function name() {}           // Declaration
const name = function() {};  // Expression
const name = () => {};        // Arrow

// Arrays
push, pop, shift, unshift, splice, slice
map, filter, reduce, find, some, every, forEach
sort((a,b) => a-b)
[...arr], [a, b] = arr

// Objects
{key: value}
obj.key, obj["key"], obj[var]
Object.keys(), Object.entries()
{...obj}, JSON.parse(JSON.stringify(obj))
for...in, {a, b} = obj
obj?.key
{ [var]: value }
```

 **Pro Tip:** Notes बनाना सिर्फ बाद में पढ़ने के लिए नहीं है। लिखने से understanding बेहतर होती है। हर topic को अपने शब्दों में लिखो!



मुझे जावास्क्रिप्ट समझ आ जाती...
TXT 964.34KB

create notes for me out of this transcript

JavaScript Complete Notes (Part 2) - DOM, Events, Forms, Storage & Projects

1. DOM (Document Object Model)

DOM क्या है?

- पूरा HTML एक **Tree Structure** है
- हर element एक **Node** है
- **DOM Manipulation** = Webpage में changes करना

Node Types

- **Element Node:** HTML tags (जैसे `<div>` , `<h1>`)
- **Text Node:** Elements के अंदर का text
- **Comment Node:** HTML comments

2. Element Selection

Methods (Modern - Recommended)

```
js
document.querySelector('h1'); // पहला element
document.querySelectorAll('.class'); // सारे elements (NodeList)
```

Methods (Legacy)

```
js
document.getElementById('id'); // ID से (hash नहीं लगाना)
document.getElementsByClassName('class'); // HTMLCollection
document.getElementsByTagName('tag');
```

Key Differences

Method	Returns	Type
querySelector	पहला element	Element
querySelectorAll	सारे elements	NodeList
getElementById	एक element	Element
getElementsByClassName	सारे elements	HTMLCollection

Pro Tip: Console में `console.dir()` use करें तो element की सारी properties दिखती हैं।

3. Text/HTML Manipulation

Property	क्या करता है
<code>innerText</code>	Visible text बदलता है
<code>textContent</code>	सारा text (hidden भी) - Faster
<code>innerHTML</code>	HTML tags के साथ बदलता है

```
js

h1.innerText = "Hello";           // Text
h1.textContent = "Hello";         // Text (faster)
h1.innerHTML = "<i>Hello</i>";     // HTML render होगा
```

4. Attribute Manipulation

```
js

element.getAttribute('href');     // Value निकालना
element.setAttribute('href', 'url'); // Value set करना
element.removeAttribute('href');  // Attribute हटाना
element.href = 'url';             // Direct change
```

Common Attributes: `src` , `href` , `title` , `disabled` , `class` , `id`

5. Style Manipulation

Direct Style

```
js

element.style.color = 'red';
element.style.backgroundColor = 'yellow';
```

Class-based (Recommended

```
js

element.classList.add('active');
element.classList.remove('active');
element.classList.toggle('active'); // लगा/हटा
element.classList.contains('active'); // Check
```

क्यों Class better है?: एक साथ कई properties change होती हैं, code clean रहता है।

6. Dynamic DOM Manipulation

Element Create करना

```
js

const div = document.createElement('div');
div.textContent = "Hello";
div.classList.add('card');
```

Element Add करना

```
js

parent.append(child);      // Last में
parent.prepend(child);     // First में
parent.insertBefore(newEl, refEl); // Specific position
```

Element Remove करना

```
js

element.remove();          // खुद को
parent.removeChild(child);  // Parent से
```

Element Replace करना

```
js

oldElement.replaceWith(newElement);
```

7. Events & Event Handling

Event क्या है?

- Browser पर कोई भी action = Event
- Click, Double Click, Mouse Move, Key Press, Input, Change, Submit, Scroll

Event Listener Add करना

```
js

element.addEventListener('click', function(e) {
  // code
});
```

Event Listener Remove करना

```
js
```

```
function handler() { }
element.addEventListener('click', handler);
element.removeEventListener('click', handler); // Same function pass करें
```

Common Events

Event	कब trigger होता है
click	Click करने पर
dblclick	Double click पर
input	Input box में type करते ही
change	Value change होने पर (select/input)
submit	Form submit होने पर
mouseover	Mouse आने पर
mouseout	Mouse जाने पर
mousemove	Mouse move होने पर
keydown	Key दबाने पर
keyup	Key छोड़ने पर

Important: Events को रटा नहीं जाता, **Google/ChatGPT** से देख लिया जाता है।

8. Event Object

Event listener के function में जो parameter आता है, वो **Event Object** है।

```
js

btn.addEventListener('click', (e) => {
  console.log(e.target); // जिस element पर event हुआ
  console.log(e.type); // Event का type
  console.log(e.preventDefault()); // Default action रोकना
});
```

Key Properties

- **target:** जिस element पर event हुआ
- **type:** Event का नाम
- **clientX, clientY:** Mouse position
- **key:** दबाई गई key (keyboard events में)
- **data:** Input value (input events में)

preventDefault()

- Form reload रोकता है
- Link navigation रोकता है
- Browser के default behavior को block करता है

9. Event Bubbling

Concept: Event जिस element पर हुआ, उससे **ऊपर (parent)** की तरफ जाता है।

Order: Child → Parent → Grandparent → ... → HTML

```
js
// Button पर click
// Button का listener → C का listener → B का listener → A का listener
```

10. Event Capturing

Concept: Event **ऊपर से नीचे** आता है (Bubbling का उल्टा)।

Enable करना: Listener के last में true लिखें।

```
js
element.addEventListener('click', handler, true);
```

Event Flow (2 Phases)

1. **Capture Phase** (Top → Bottom) - true वाले listeners
2. **Bubble Phase** (Bottom → Top) - Normal listeners

Example Order

```
text
A (capture: true) → C (capture: true) → Button → C → B → A
```

11. Event Delegation

Concept: Parent पर एक listener लगाओ, child पर नहीं।

फायदा: हर child पे listener नहीं लगाना पड़ता।

```
js
ul.addEventListener('click', (e) => {
  if (e.target.tagName === 'LI') {
    e.target.classList.toggle('done');
  }
});
```

12. Forms & Validation

Form से Values पढ़ना

```
js

const input = document.querySelector('#email');
const value = input.value;           // Input की value
const content = div.textContent;     // Text content
```

Form Submit रोकना

```
js

form.addEventListener('submit', (e) => {
  e.preventDefault(); // ReLoad रोके
  // Validation Logic
});
```

Inline Validation (HTML Attributes)

Attribute	काम
required	Field भरना जरूरी
minlength	Minimum characters
maxlength	Maximum characters
pattern	Regex pattern
type="email"	Email format check

Note: इन attributes को **Inspect Element** से bypass किया जा सकता है, इसलिए JS validation भी जरूरी है।

Regex Validation

```
js

const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const isValid = emailRegex.test(email);
```

Pro Tip: Regex खुद नहीं लिखते, ChatGPT/Google से लेते हैं।

Value vs Text Content

- `.value` : Input elements के लिए
- `.textContent` : Div, p, h1 जैसे text elements के लिए

13. Timers & Intervals

setTimeout - एक बार चलता है

```
js

const timer = setTimeout(() => {
  console.log("Hello");
}, 3000); // 3 seconds

clearTimeout(timer); // रोकना
```

setInterval - बार-बार चलता है

```
js

const interval = setInterval(() => {
  console.log("Hello");
}, 1000); // हर 1 second

clearInterval(interval); // रोकना
```

Use Cases

- **setTimeout**: Notification auto-hide, Delayed action
- **setInterval**: Countdown, Live clock, Progress bar

Countdown Example

```
js

let count = 10;
const interval = setInterval(() => {
  console.log(count--);
  if (count < 0) clearInterval(interval);
}, 1000);
```

14. Local Storage

क्या है?

- Browser का **Database**
- Data **हमेशा** रहता है (Browser बंद करने पर भी)
- सिर्फ **Strings** store करता है
- Capacity: ~5 MB

Methods

```
js
```

```

localStorage.setItem('name', 'Harsh'); // Save
localStorage.getItem('name'); // Get
localStorage.removeItem('name'); // Remove एक
localStorage.clear(); // सब हटाओ

```

Object/Array Store करना

```

js

// Store
localStorage.setItem('users', JSON.stringify(users));

// Retrieve
const users = JSON.parse(localStorage.getItem('users'));

```

15. Session Storage

क्या है?

- Local Storage जैसा ही है
- Tab बंद होते ही data चला जाता है
- Temporary data के लिए

Methods

- Same as Local Storage: `setItem`, `getItem`, `removeItem`, `clear`

Use Cases

- Temporary tokens
- Form data during session
- Cart during session

16. Cookies

क्या है?

- Browser में छोटा data store करने के लिए
- Capacity: ~4 KB (Local Storage से बहुत कम)
- हर page reload पर Server को जाता है
- Browser बंद करने पर भी रहता है

Methods

```
js
```

```
document.cookie = "email=harsh@test.com"; // Set
document.cookie // Get (सारी cookies)
```

Cookie Expiry

```
js
document.cookie = "name=Harsh; expires=Fri, 31 Dec 2027 23:59:59 GMT";
```

Cookie देखने के लिए

- Chrome Extension: **EditThisCookie**
- Browser DevTools → Application → Cookies

Use Cases

- Login tokens
- User preferences
- Tracking

17. Comparison: Local vs Session vs Cookie

Feature	Local Storage	Session Storage	Cookie
Capacity	~5 MB	~5 MB	~4 KB
Expiry	Never	Tab close	Manual
Server को जाता है	✗	✗	✓
Data Type	String	String	String

18. Projects Built

Project 1: Real-Time Search Filter

Features:

- सारे users show करना
- Input में type करते ही filter
- Case-insensitive search
- Debouncing (performance)

Key Concepts:

- Array filter
- `startsWith()` method

- Event delegation
- DOM manipulation

js

```
users.filter(user =>
  user.name.toLowerCase().startsWith(input.value.toLowerCase())
);
```

Project 2: Notes/Call Reminder App

Features:

- Cards create करना
- Local storage में save
- Form validation
- Priority filters
- Circular card stack animation
- Up/Down buttons

Key Concepts:

- `document.createElement()`
- Form validation
- LocalStorage (JSON.stringify/parse)
- Class manipulation
- z-index manipulation

19. Common Confusions

1. innerText vs textContent vs innerHTML

- **innerText**: Visible text, slower
- **textContent**: सारा text (hidden भी), faster ✔️
- **innerHTML**: HTML tags के साथ

2. Events vs Listeners

- **Event**: Action जो हुआ (click, mouseover)
- **Listener**: Function जो action का response देता है

3. Bubbling vs Capturing

- **Bubbling**: Bottom → Top (Default)
- **Capturing**: Top → Bottom (`true` parameter)

4. value vs textContent

- **value:** Input elements के लिए
- **textContent:** HTML text elements के लिए

5. Local Storage में सिर्फ Strings

- Arrays/Objects store करने हों तो `JSON.stringify()` use करें
- Retrieve करते समय `JSON.parse()` use करें

6. Splice vs Slice

- **Splice:** Original array change करता है
- **Slice:** नया array return करता है

7. defer vs async

- **defer:** HTML parse होने के बाद script run हो
- **async:** Script load होते ही run हो

20. Important Tips

Google/ChatGPT Skills

- Events के नाम रटा नहीं जाते
- Regex खुद नहीं लिखते
- Property names भूल जाएं तो search करें

Debugging

- `console.dir()` सारी properties दिखाता है
- `console.log()` कभी tag format में दिखाता है
- Errors को धैर्य से पढ़ें

Code Quality

- **Class-based styling** prefer करें (inline style से बेहतर)
- **Event delegation** use करें (multiple listeners से बेहतर)
- **Debouncing** use करें (performance के लिए)
- **Early return pattern** use करें (deep if-else से बेहतर)

Browser Data देखने के लिए

- DevTools → Application → Local Storage / Session Storage / Cookies

21. Quick Reference Card

```

js

// Selection
document.querySelector('.class');
document.querySelectorAll('div');
document.getElementById('id');

// Manipulation
el.textContent = "text";
el.innerHTML = "<b>text</b>";
el.setAttribute('src', 'url');
el.classList.add('active');
el.classList.toggle('active');
el.style.color = 'red';

// Create/Add/Remove
const div = document.createElement('div');
parent.append(div);
parent.prepend(div);
parent.insertBefore(newEl, ref);
el.remove();

// Events
el.addEventListener('click', (e) => {
  e.preventDefault();
  console.log(e.target);
});

// Event Bubbling
el.addEventListener('click', handler); // Bubble
el.addEventListener('click', handler, true); // Capture

// Local Storage
localStorage.setItem('key', 'value');
localStorage.getItem('key');
localStorage.removeItem('key');
localStorage.clear();

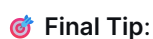
// JSON
JSON.stringify(obj); // Object → String
JSON.parse(str);    // String → Object

// Timers
setTimeout(fn, 3000);
setInterval(fn, 1000);
clearTimeout(id);
clearInterval(id);

// Form
form.addEventListener('submit', (e) => {
  e.preventDefault();
  const value = input.value;
});

// Regex
const regex = /pattern/;
regex.test(string); // true/false

```



Final Tip:

- **Notes बनाओ** (लिखने से understanding बेहतर होती है)
- **Projects बनाओ** (practice से confidence आता है)
- **Google/ChatGPT use करना सीखो** (हर कोई करता है)
- **Consistency रखो** (daily practice से ही mastery आती है)

Remember: "You can't memorize all of it. Nobody does. You just need to know how to approach problems and where to find answers."



हेलो एवरीवन। तो ये शेरियंस ...
TXT 515.18KB

create notes for me out of this transcript

JavaScript Advanced - Complete Notes

1. Scope & Execution Context

Scope क्या है?

- **Scope** = आपके created variables और functions कहां तक use हो सकते हैं

3 Types of Scope

Scope	Meaning	Example
Global	पूरे code में accessible	Function/Block के बाहर
Function	सिर्फ function के अंदर	var का scope
Block	सिर्फ {} के अंदर	let , const , if/while

Execution Context

- **Hypothetical box** जो function देखते ही बनता है
- **2 Phases:**
 1. **Memory Phase:** Variables के लिए space allocate होती है
 2. **Execution Phase:** Code चलता है

Abstract Concept: जैसे हवा दिखती नहीं, पर होती है — वैसे ही Execution Context दिखता नहीं, पर होता है।

2. Lexical vs Dynamic Scoping

Lexical Scoping (JavaScript Follow करती है)

- **Physical position** matter करती है
- आप कहां लिखे हो, यह तय करता है कि क्या access कर सकते हो

Dynamic Scoping (JavaScript नहीं follow करती)

- Calling location matter करती है
- कहां से call कर रहे हो, इस पर depend करता है

Example:

```
js

let a = 12;
function abc() {
  console.log(a); // Lexical: 12 (global से)
}
function dfg() {
  let a = 13;
  abc(); // Dynamic होती तो 13 आता
}
dfg(); // Output: 12 (Lexical scoping)
```

3. Closures

Definition

- Function जो return करे एक और function
- Returning function parent function का कोई variable use करे

Variables कैसे Preserve होते हैं?

- Function खत्म होने पर उसके variables खत्म हो जाते हैं
- लेकिन closure बनने पर JavaScript एक **Backlink** बनाता है
- `[[Environment]]` में function और variables save हो जाते हैं

Use Cases

1. **Private Counters:** Counter जो अपनी value याद रखे
2. **Encapsulation:** Inner logic hide करना
3. **Private Variables:** बाहर से change नहीं कर सकते

Private Counter Example

```
js

function countForMe() {
  let c = 0;
  return function() {
    if (c < 5) {
      c++;
      console.log(c + " times clicked");
    } else {
      console.log("Limit exceeded");
    }
  }
};
```

```

}
const counter = countForMe();
counter(); // 1
counter(); // 2

```

Private Variables (Encapsulation)

```

js

function clickLimiter() {
  let clicks = 0;
  return function() {
    if (clicks < 5) {
      clicks++;
      console.log(clicks + " clicks");
    } else {
      console.log("Limit exceeded");
    }
  };
}
// clicks को बाहर से change नहीं कर सकते

```

4. Closures से Project: Toast Notifications

Features

- Custom position (top-left, top-right, bottom-left, bottom-right)
- Theme (dark/light)
- Duration control
- Multiple notifications stack

Implementation

```

js

function createToast(config) {
  // Default configuration
  const finalConfig = {
    positionX: config.positionX || "right",
    positionY: config.positionY || "top",
    theme: config.theme || "dark",
    duration: config.duration || 3
  };

  return function(text) {
    const div = document.createElement("div");
    div.classList.add("toast");
    div.textContent = text;

    // Position styling
    if (finalConfig.positionX === "right") {
      div.classList.add("right-10");
    } else {
      div.classList.add("left-10");
    }
  }
}

```

```
if (finalConfig.positionY === "top") {
  div.classList.add("top-10");
} else {
  div.classList.add("bottom-10");
}

// Theme styling
if (finalConfig.theme === "dark") {
  div.classList.add("bg-black", "text-white");
} else {
  div.classList.add("bg-gray-100", "text-black");
}

// Parent में append
document.querySelector("#parent").appendChild(div);

// Auto remove
setTimeout(() => {
  document.querySelector("#parent").removeChild(div);
}, finalConfig.duration * 1000);
};
}

const toaster = createToast({
  positionX: "right",
  positionY: "top",
  theme: "dark",
  duration: 3
});

toaster("Download Done");
toaster("Harsh Accepted Your Request");
```

5. this Keyword

Different Contexts में this की Value

Context	this की Value
Global Scope	window
Function (Normal)	window
Method (ES5 Function)	Object जिसका method है
Method (ES6 Arrow)	Parent से लेता है (usually window)
Event Handler	Element जिस पर listener लगा है
Class	Blank Object (new से बना)

Important Rules

- **Arrow function** हमेशा this अपने parent से लेता है
- **Method में Arrow function** use करने से this खो जाता है (window हो जाता है)

- Method में ES5 Function use करने से this object होता है

Method में Nested Function

```
js

const obj = {
  name: "Harsh",
  fn: function() {
    console.log(this); // obj

    function inner() {
      console.log(this); // window (खो गया)
    }
    inner();

    const innerArrow = () => {
      console.log(this); // obj (parent से लिया)
    };
    innerArrow();
  }
};
```

6. Manual Binding: call, apply, bind

Function Call करते वक्त this की Value सेट करना

Method	क्या करता है	Parameters
call	Function को चलाता है	(obj, arg1, arg2, ...)
apply	Function को चलाता है	(obj, [arg1, arg2, ...])
bind	नया function return करता है	(obj, arg1, arg2, ...)

Examples

```
js

function abcd(a, b, c) {
  console.log(this, a, b, c);
}

const obj = { name: "Harsh" };

abcd.call(obj, 1, 2, 3); // Function चला
abcd.apply(obj, [1, 2, 3]); // Function चला
const fnc = abcd.bind(obj, 1, 2, 3); // Function नहीं चला
fnc(); // अब चलेगा
```

bind का Use Case

- Event listener में this fix करना

- Function को बाद में call करने के लिए ready करना

7. Object-Oriented JavaScript

Constructor Function

```
js

function CreatePencil(name, price, color, company) {
  this.name = name;
  this.price = price;
  this.color = color;
  this.company = company;
}

const pencil1 = new CreatePencil("Nataraj", 10, "black", "Nataraj");
const pencil2 = new CreatePencil("Doms", 10, "red", "Doms");
```

new Keyword क्या करता है?

1. Blank Object बनाता है
2. this को blank object से replace करता है
3. Function run करता है
4. Object return करता है

Prototypes

- Shared Memory जैसा होता है
- Constructor function के prototype पर field add करें
- सब instances को automatically मिल जाती है

```
js

CreatePencil.prototype.company = "Sheryians";
CreatePencil.prototype.write = function(text) {
  const h1 = document.createElement("h1");
  h1.textContent = text;
  h1.style.color = this.color;
  document.body.appendChild(h1);
};

pencil1.write("Hello"); // Black color
pencil2.write("World"); // Red color
```

फायदा: Memory बचती है, हर instance का अपना method नहीं होता

8. Classes (ES6)

Class Syntax

js

```

class CreatePencil {
  constructor(name, price, color, company) {
    this.name = name;
    this.price = price;
    this.color = color;
    this.company = company;
  }

  write(text) {
    const h1 = document.createElement("h1");
    h1.textContent = text;
    h1.style.color = this.color;
    document.body.appendChild(h1);
  }

  erase() {
    document.querySelectorAll("h1").forEach(el => el.remove());
  }
}

const p1 = new CreatePencil("Nataraj", 10, "black", "Nataraj");
const p2 = new CreatePencil("Doms", 10, "red", "Doms");

p1.write("Hello"); // Black
p2.write("World"); // Red
p1.erase(); // सारे हटा देगा

```

Class Features

- Constructor **first** होता है
- Methods constructor के बाहर
- `class` keyword use करते हैं
- First letter **Capital** होता है

9. Inheritance: extends & super

Classical Inheritance

js

```

class User {
  constructor(name, address, username, email) {
    this.name = name;
    this.address = address;
    this.username = username;
    this.email = email;
    this.role = "user";
  }

  write(text) {
    const h1 = document.createElement("h1");
    h1.textContent = `${this.name}: ${text}`;
    document.body.appendChild(h1);
  }
}

```

```

    checkRole() {
        return "You are a " + this.role;
    }
}

class Admin extends User {
    constructor(name, address, username, email) {
        super(name, address, username, email);
        this.role = "admin";
    }

    remove() {
        document.querySelectorAll("h1").forEach(el => el.remove());
    }
}

const admin = new Admin("Harsh", "Bhopal", "harsh123", "h@h.com");
admin.write("Hello"); // write method मिला (inherited)
admin.remove(); // admin का अपना method
admin.checkRole(); // "You are a admin"

```

super क्या करता है?

- Parent class के constructor को call करता है
- Parent की properties inherit करता है

10. Prototypal vs Classical Inheritance

Feature	Classical	Prototypal
Languages	Java, C++	Only JavaScript
Between	Class → Class	Object → Object
Method	extends	Object.create()

Prototypal Inheritance Example

```

js

const coffee = {
    color: "dark",
    drink: function() {
        console.log("Good Good Good");
    }
};

const cappuccino = Object.create(coffee);
// cappuccino के पास coffee की सारी properties हैं
cappuccino.taste = "bitter";

cappuccino.drink(); // "Good Good Good"

```

11. Synchronous vs Asynchronous

Synchronous Code

- Line by line चलता है
- हर line complete होने के बाद अगली चलेगी

Asynchronous Code

- जब ready हो तब चले
- बीच में दूसरा code चल सकता है

Example

```
js

console.log("A1");
setTimeout(() => console.log("A3"), 2000);
console.log("A2");
// Output: A1, A2, A3
```

12. Callbacks

Callback क्या है?

- Function जो दूसरे function को parameter के रूप में दिया जाए

Example

```
js

function getProfileData(username, callback) {
  setTimeout(() => {
    callback({ username: username, age: 26 });
  }, 2000);
}

getProfileData("Harsh", (data) => {
  console.log(data); // { username: "Harsh", age: 26 }
});
```

13. Callback Hell

Problem

- एक callback के अंदर दूसरा callback
- दूसरे के अंदर तीसरा callback
- Christmas Tree जैसा pattern बन जाता है

Example

```
js

getProfileData("Harsh", (profileData) => {
  getAllPosts(profileData._id, (posts) => {
    getSavedPosts(profileData._id, (savedPosts) => {
      // और भी nested...
    });
  });
});
```

Solution

- Promises
- Async/Await

14. Promises

Promise क्या है?

- Async operation का result
- 3 States: Pending, Fulfilled, Rejected

Promise बनाना

```
js

const pr = new Promise((resolve, reject) => {
  setTimeout(() => {
    const randomNum = Math.floor(Math.random() * 10);
    if (randomNum > 5) {
      resolve(randomNum); // Fulfilled
    } else {
      reject(randomNum); // Rejected
    }
  }, 3000);
});

pr.then((val) => {
  console.log("Resolved with", val);
}).catch((err) => {
  console.log("Rejected with", err);
});
```

Promise Methods

- .then() : जब resolve हो
- .catch() : जब reject हो
- .finally() : हमेशा चले

Promise चलाने का Pattern

1. Promise बनाओ (जो async operation represent करें)
2. `.then()` में success handle करें
3. `.catch()` में error handle करें

15. Async/Await

Promise का Cleaner Version

```
js

async function abcd() {
  try {
    const val = await pr;
    console.log("Resolved with", val);
  } catch (err) {
    console.log("Rejected with", err);
  }
}

abcd();
```

Key Points

- async function के अंदर ही await use कर सकते हैं
- await प्रॉमिस के result का wait करता है
- try/catch से error handle करते हैं

16. Fetch API

Fetch क्या है?

- JavaScript का built-in function
- URL से data मंगाता है
- Promise-based

Basic Fetch

```
js

fetch("https://api.example.com/users/3")
  .then((rawData) => rawData.json())
  .then((data) => console.log(data))
  .catch((err) => console.log(err));
```

Raw Data को JSON में Convert करना जरूरी

- पहली बार में Readable Stream मिलती है
- पढ़ने के लिए `.json()` use करना पड़ता है

Fetch के साथ Project: User Cards

```
js

function getUsers() {
  fetch("https://randomuser.me/api/?results=3")
    .then((rawData) => rawData.json())
    .then((data) => {
      document.querySelector("#users").innerHTML = "";

      data.results.forEach(user => {
        const card = document.createElement("div");
        card.innerHTML = `
          
          <h3>${user.name.first} ${user.name.last}</h3>
          <p>${user.email}</p>
        `;
        document.querySelector("#users").appendChild(card);
      });
    });
}

getUsers();
document.querySelector("#refresh").addEventListener("click", getUsers);
```

17. HTTP Basics (Backend Overview)

HTTP Status Codes

- 200-299: Success
- 400-499: Client errors (404 not found, 401 unauthorized)
- 500-599: Server errors

HTTP Methods

- GET: Data लेना
- POST: Data भेजना
- PUT/PATCH: Update करना
- DELETE: Delete करना

JSON Format

- JavaScript Object Notation
- Both sides double quotes होते हैं
- API responses का standard format

18. Form Submission via Fetch

Basic Pattern

js

```

form.addEventListener("submit", (e) => {
  e.preventDefault(); // Page reload रोको

  fetch("https://api.example.com/submit", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      name: nameInput.value,
      email: emailInput.value,
      password: passwordInput.value
    })
  })
  .then(res => res.json())
  .then(data => console.log(data))
  .catch(err => console.error(err));
});

```

Note: यह backend-dependent topic है, backend के बिना practice नहीं कर सकते

19. Important Patterns

Encapsulation with Closures

js

```

function clickLimiter() {
  let clicks = 0;
  return function() {
    if (clicks < 5) {
      clicks++;
      console.log(clicks);
    } else {
      console.log("Limit exceeded");
    }
  };
}
// clicks private है, बाहर से change नहीं कर सकते

```

Factory Pattern

js

```

function CreatePencil(name, price, color, company) {
  return {
    name, price, color, company,
    write(text) {
      // ...
    }
  };
}

```

Prototype for Shared Memory

```
js

CreatePencil.prototype.write = function(text) {
  // सारे instances को common method
};
```

20. Common Confusions

Topic	Confusion	Answer
Scope	Global vs Function vs Block	var = function, let/const = block
this	कहाँ क्या value	Global = window, Method = obj, Arrow = parent
call vs apply	क्या फर्क	call = multiple args, apply = array
bind vs call	क्या फर्क	bind = new function देता है, call = चला देता है
Promise vs Callback	कब use करें	Promise = cleaner, Callback = older
Sync vs Async	क्या फर्क	Sync = line by line, Async = when ready
Class vs Function	क्या फर्क	Class = syntactic sugar, Function = older way

21. Practice Projects

Project 1: Toast Notifications

Concepts: Closures, createElement, setTimeout, classList

Project 2: User Manager

Concepts: Classes, this, bind, methods, encapsulation

Project 3: User Cards with API

Concepts: Fetch, JSON, forEach, createElement, refresh button

22. Quick Reference Card

```
js
```

```
// Scope
var x;      // Function scope
let y;      // Block scope
const z;    // Block scope

// this values
function fn() { console.log(this); } // window
const obj = {
  fn: function() { console.log(this); } // obj
};
const obj2 = {
  fn: () => console.log(this) // window (parent)
};

// Binding
fn.call(obj, 1, 2);      // Runs with obj
fn.apply(obj, [1, 2]);   // Runs with obj
const newFn = fn.bind(obj); // Returns new function
newFn();                 // Now runs

// Constructor
function User(name) {
  this.name = name;
}
User.prototype.write = function() { };
const u = new User("Harsh");

// Class
class User {
  constructor(name) {
    this.name = name;
  }
  write() { }
}
class Admin extends User {
  constructor(name) {
    super(name);
  }
}

// Prototypal
const a = { x: 1 };
const b = Object.create(a); // b inherits a

// Promise
const p = new Promise((resolve, reject) => {
  setTimeout(() => resolve(12), 1000);
});
p.then(val => console.log(val)).catch(err => console.log(err));

// Async/Await
async function fn() {
  try {
    const val = await p;
    console.log(val);
  } catch (err) {
    console.log(err);
  }
}

// Fetch
fetch(url)
```

```
.then(raw => raw.json())
.then(data => console.log(data))
.catch(err => console.log(err));
```

23. Key Takeaways

1. **Closures** = Function + Variables जो याद रहते हैं
2. **this** = Context पर depend करता है
3. **call/apply/bind** = Manual binding के लिए
4. **new** = Blank object बनाता है
5. **Prototypes** = Shared memory
6. **Class** = Constructor का clean version
7. **extends/super** = Inheritance के लिए
8. **Promise** = Async result handle करने का तरीका
9. **Async/Await** = Promise का cleaner syntax
10. **Fetch** = API से data लाने का तरीका

Remember:

- **Code रटा नहीं जाता**, approach समझनी होती है
- **Google/ChatGPT** use करना कोई शर्म नहीं
- **Project बनाओ** practice के लिए
- **Consistency** सबसे important है

24. Practice Checklist

- ☐ Scope समझ लो (Global, Function, Block)
- ☐ Execution Context की mental model बना लो
- ☐ Closures practice करो (Counter, Encapsulation)
- ☐ Toast Notification project बनाओ
- ☐ **this** के सारे contexts practice करो
- ☐ **call, apply, bind** practice करो
- ☐ User Manager project बनाओ
- ☐ Constructor Function समझ लो
- ☐ Prototypes समझ लो
- ☐ Classes समझ लो
- ☐ **extends** और **super** समझ लो
- ☐ Prototypal Inheritance practice करो
- ☐ Callbacks समझ लो
- ☐ Callback Hell देख लो
- ☐ Promises समझ लो
- ☐ Async/Await समझ लो

- Fetch API practice करो
- User Cards project बनाओ

Final Tip: जो topic समझ ना आए, उसे **3-4 बार देखो**। Code लिखो, break करो, debug करो। Practical से ही आएगा।



जावास्क्रिप्ट 4 में आपका स्वाग...
TXT 366.88KB

create notes for me out of this transcript

JavaScript Advanced - Complete Notes (Part 4)

Section 1: Design Patterns in JavaScript

डिज़ाइन पैटर्न क्यों जरूरी है?

- कॉलेज स्टूडेंट vs असली इंजीनियर में फर्क = Design Patterns
- **Maintainable, Scalable, Modular** code लिखने के लिए
- लाखों लाइन के code को manage करने के लिए
- Public/Private logic को separate रखने के लिए

1. Module Pattern (IIFE Based)

Definition: अपना code एक **Self-Executing Function (IIFE)** के अंदर लिखना ताकि variables और functions **private** रहें।

IIFE क्या है?

```
js

(function() {
  // code
})();
```

Module Pattern Example:

```
js

const Bank = (function() {
  let bankBalance = 12000;

  function checkBalance() {
    console.log(bankBalance);
  }

  function setBalance(val) {
    bankBalance = val;
  }

  function withdraw(amount) {
    if (amount <= bankBalance) {
      bankBalance -= amount;
    }
  }
})();
```

```

        console.log("Withdrew", amount, "New balance:", bankBalance);
    }
}

// सिर्फ यही बाहर accessible
return {
    checkBalance,
    setBalance,
    withdraw
};
})();

Bank.checkBalance(); // 12000
Bank.withdraw(11000); // Withdrew 11000
Bank.checkBalance(); // 1000
// Bank.bankBalance // ❌ Error (private)

```

Key Points:

- IIFE के अंदर जो भी बनाओ → **Private**
- Return में object भेजो → **Public**
- Bank Balance जैसी sensitive चीजें private रहती हैं

2. Revealing Module Pattern

Definition: Module Pattern जैसा ही, बस return करते वक्त **names बदल सकते हो।**

```

js

const Bank = (function() {
    let bankBalance = 12000;

    function checkBalance() { console.log(bankBalance); }
    function setBalance(val) { bankBalance = val; }
    function withdraw(amount) { /* ... */ }

    // नाम बदल के भेजो
    return {
        check: checkBalance, // checkBalance → check
        set: setBalance, // setBalance → set
        draw: withdraw // withdraw → draw
    };
})();

Bank.check(); // 12000
Bank.set(5000);
Bank.draw(1000);

```

फर्क: Module Pattern में same name, Revealing में rename कर सकते हो।

3. Factory Function Pattern

Definition: एक function जो **नए objects create करके return** करता है (बिना `new` keyword या `class` के)।

```

js

function createProduct(name, price) {
  let stock = 10;

  return {
    name,
    price,

    buy(quantity) {
      if (quantity <= stock) {
        stock -= quantity;
        console.log(quantity, "pieces booked.", stock, "pieces left");
      } else {
        console.error("Out of stock! Only", stock, "pieces left");
      }
    },

    refill(quantity) {
      stock += quantity;
      console.log("Refilled! Now", stock, "pieces available");
    },

    checkStock() {
      console.log("Current stock:", stock);
    }
  };
}

const iPhone = createProduct("iPhone", 70000);
const KitKat = createProduct("KitKat", 10);

iPhone.buy(6);           // 6 pieces booked, 4 Left
KitKat.buy(5);           // 5 pieces booked, 5 Left
iPhone.checkStock();     // 4 pieces
KitKat.checkStock();     // 5 pieces

```

Key Points:

- हर object का **अपना memory/data** होता है
- iPhone और KitKat का stock अलग-अलग
- Class/Constructor के बिना object creation

4. Observer Pattern (YouTube Style)

Definition: एक Object (Subject) की state change होने पर सारे dependents (Observers) को notify करना।

```

js

class YouTubeChannel {
  constructor() {
    this.subscribers = [];
  }

  subscribe(user) {
    this.subscribers.push(user);
    user.update("You have subscribed the channel");
  }
}

```

```

    }

    unsubscribe(user) {
      this.subscribers = this.subscribers.filter(sub => sub !== user);
      user.update("You have unsubscribed the channel");
    }

    notify(message) {
      this.subscribers.forEach(sub => {
        sub.update(`${sub.name}, ${message}`);
      });
    }
  }
}

class User {
  constructor(name) {
    this.name = name;
  }

  update(message) {
    console.log(message);
  }
}

// Usage
const sheryians = new YouTubeChannel();
const harsh = new User("Harsh");
const amit = new User("Amit");

sheryians.subscribe(harsh);
// "You have subscribed the channel"

sheryians.subscribe(amit);
// "You have subscribed the channel"

sheryians.notify("New video is live on the channel");
// "Harsh, New video is live on the channel"
// "Amit, New video is live on the channel"

sheryians.unsubscribe(harsh);
// "You have unsubscribed the channel"

```

Key Points:

- YouTube Subscribe Model
- Subject (Channel) → Observers (Users)
- State change पर सबको notify करना
- **React, Redux** में भी use होता है

Section 2: Performance Optimization

क्यों जरूरी है?

- **User Experience** = Return Users
- 2-3 सेकंड में attention grab नहीं किया तो user चला जाएगा
- Company को Loss

1. Debouncing vs Throttling

Debouncing

Definition: आपके actions के बीच में जो specific time gap चाहिए, जब उतना delay आएगा तभी reaction होगा।

Example - Amazon Search:

text

आप "BULB" type कर रहे हो,
हर letter पर search नहीं होता।
जब आप typing रोकते हो → तब search होता है।

js

```
const input = document.querySelector("input");

function debounce(fnc, delay) {
  let timer;

  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => {
      fnc(...args);
    }, delay);
  };
}

input.addEventListener("input", debounce(function() {
  console.log("Search triggered");
}, 1000)); // 1 second delay
```

कब होता है: Typing, Search, Window Resize

Throttling

Definition: हर specific interval पर event चलेगा, चाहे action कितना भी हो।

Example - Mouse Move:

text

Mouse continuously move कर रहा है,
हर 1 second में एक बार position print होगी।
अगर mouse रुका → throttle भी रुकेगा।

js

```
function throttle(fnc, delay) {
  let last = 0;

  return function(...args) {
    const now = Date.now();
```

```
    if (now - last >= delay) {
      last = now;
      fnc(...args);
    }
  };
}

input.addEventListener("input", throttle(function() {
  console.log("Triggered");
}, 1000));
```

कब होता है: Scroll, Mouse Move, Gaming controls

Debounce vs Throttle:

Feature	Debounce	Throttle
कब चले	Actions के बाद gap आए	हर interval पर
कब रुके	Action पर हर बार timer reset	हमेशा चलता रहे
Example	Search bar	Mouse position tracker
Frequency	Low (end पर)	Regular interval

2. Lazy Loading Images (Intersection Observer)

Concept: जो images view में हैं सिर्फ वो load करो, बाकी नीचे वाली images scroll करने पर load होंगी।

```
html




css

.lazy-image {
  opacity: 0;
  transition: opacity 0.5s;
}

.lazy-image.loaded {
  opacity: 1;
}

js

const images = document.querySelectorAll(".lazy-image");

const observer = new IntersectionObserver((entries, obs) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const image = entry.target;
```

```

        image.src = image.dataset.src;
        image.classList.add("loaded");
        obs.unobserve(image);
    }
});
}, {
    root: null,        // viewport
    threshold: 0.1     // 10% visible होने पर trigger
});

images.forEach(image => observer.observe(image));

```

Key Points:

- data-src में real image path
- View में आने पर src set करो
- Intersection Observer API use करो
- Page Load Time कम होता है

3. Avoiding Unnecessary Reflows & Repaints

Problem: हर DOM change पर browser को **Reflow** और **Repaint** करना पड़ता है। ये slow है।

Bad Practice:

```

js

for (let i = 0; i < 10000; i++) {
    const li = document.createElement("li");
    li.textContent = i;
    ul.appendChild(li); // ❌ हर बार DOM tree refresh
}

```

Good Practice (Document Fragment):

```

js

const fragment = document.createDocumentFragment();

for (let i = 0; i < 10000; i++) {
    const li = document.createElement("li");
    li.textContent = i;
    fragment.appendChild(li); // ✅ Memory में add
}

ul.appendChild(fragment); // ✅ एक बार DOM में add

```

CSS में भी:

- ❌ एक-एक property बदलना
- ✅ Class add/remove करना

4. Memory Leaks (Timers & Event Listeners)

Problem: Timer या listener clear नहीं करने पर memory में space occupy रहता है।

Bad Example:

```
js

let count = 0;
const interval = setInterval(() => {
  if (count < 10) {
    count++;
    console.log(count);
  }
  // ❌ Interval clear नहीं हुआ
}, 1000);
```

Good Example:

```
js

let count = 0;
const interval = setInterval(() => {
  if (count < 10) {
    count++;
    console.log(count);
  } else {
    clearInterval(interval); // ✅ Timer clear
    console.log("Finished");
  }
}, 1000);
```

Rules:

- हर setInterval को clearInterval करो
- हर Event Listener को removeEventListener करो (जब जरूरत न हो)

5. Code Splitting (Dynamic Imports)

Concept: पूरे code को एक साथ load मत करो। जरूरत पड़ने पर load करो।

HTML:

```
html

<script type="module" src="script.js"></script>
```

script.js:

```
js

document.querySelector("#btn").addEventListener("click", async () => {
  const { veryHeavy } = await import("./heavy.js");
  veryHeavy();
});
```

heavy.js:

```
js

export function veryHeavy() {
  console.log("Joint feature loaded");
  alert("Heavy feature loading complete");
}
```

Key Points:

- type="module" जरूरी है
- import() dynamic loading
- await use करना पड़ेगा
- async function में होना चाहिए
- Initial load fast होता है

Section 3: Advanced Topics & Architecture Thinking

1. Separation of Concerns

Definition: DOM का code अलग, Logic का code अलग रखना।

Bad Code:

```
js

btn.addEventListener("click", () => {
  const num1 = Math.floor(Math.random() * 10);
  const num2 = Math.floor(Math.random() * 10);
  const addition = num1 + num2; // ❌ Logic mixed with DOM

  const li = document.createElement("li");
  li.textContent = addition;
  ul.appendChild(li);
});
```

Good Code:

```
js

function performAddition(a, b) {
  return a + b;
}

btn.addEventListener("click", () => {
  const num1 = Math.floor(Math.random() * 10);
  const num2 = Math.floor(Math.random() * 10);
  const addition = performAddition(num1, num2); // ✅ Logic separate

  const li = document.createElement("li");
  li.textContent = addition;
  ul.appendChild(li);
});
```

फायदा:

- Logic को test कर सकते हो
- Logic को reuse कर सकते हो
- Code readable होता है
- **React** में भी यही concept

2. Custom Utilities: अपना Map Function

Map क्या करता है?

- Array पर चलता है
- हर element पर callback function call करता है
- New array return करता है

अपना Map Function:

```
js

function myMap(arr, callback) {
  const newArr = [];

  for (let i = 0; i < arr.length; i++) {
    newArr.push(callback(arr[i], i, arr));
  }

  return newArr;
}

// Usage
const arr = [1, 2, 3, 4];
const result = myMap(arr, (val) => val + 2);
console.log(result); // [3, 4, 5, 6]
```

Key Points:

- Blank array बनाओ
- Loop चलाओ
- Callback को value, index, array pass करो
- Result को push करो
- Return करो

3. Deep Clone (Reference)

Note: Deep Clone और Shallow Copy का पूरा 45 मिनट का video है। वहीं से सीखें।

Summary:

- Shallow Copy: Top level values copy, Nested objects reference
- Deep Copy: सब कुछ असली copy

```
js
```

```
// Shallow
const shallowCopy = { ...original };

// Deep
const deepCopy = JSON.parse(JSON.stringify(original));
```

Section 4: How JS Works in Browser

1. Call Stack

JavaScript Single-Threaded है - एक बार में एक ही काम।

Call Stack क्या है?

- Functions को call करने पर stack में add होते हैं
- Function complete होने पर pop होते हैं
- **LIFO** (Last In First Out)

Example:

```
js

function A() {
  console.log("A");
}

function B() {
  A();
  console.log("B");
}

function C() {
  B();
  console.log("C");
}

C();
// Output: A, B, C
```

Step by Step:

1. C() call → Stack: [C]
2. C ने B() call → Stack: [C, B]
3. B ने A() call → Stack: [C, B, A]
4. A print "A" → Pop → Stack: [C, B]
5. B print "B" → Pop → Stack: [C]
6. C print "C" → Pop → Stack: []

2. Web APIs

Web APIs क्या हैं?

- Browser द्वारा JavaScript को दिए जाने वाले features
- JS Language का part नहीं है
- जैसे: `setTimeout`, `setInterval`, `alert`, `prompt`, `console`, `localStorage`

Browser vs JS:

text

JavaScript Language: 400 features
 Browser Web APIs: 100 features
 Total: 500 features (user के लिए available)

Demo: Console में `window` type करो - सारी Web APIs दिखेंगी।

Node.js में: ये Web APIs Directly काम नहीं करतीं, Node का अपना environment है।

3. Event Loop

दो प्रकार के Tasks:

- **Synchronous:** एक के बाद एक (line by line)
- **Asynchronous:** सब एक साथ चलो, जो पहले complete हो पहले चले

Example:

js

```
setTimeout(() => console.log("A"), 3000);
setTimeout(() => console.log("B"), 1000);
setTimeout(() => console.log("C"), 2000);
// Output: B, C, A (time के हिसाब से)
```

Event Loop का Process:

1. **Call Stack:** जो भी function चल रहा है
2. **Web APIs:** जो async operations run हो रहे हैं
3. **Task Queue (Callback Queue):** जो async operations complete हो गए हैं
4. **Event Loop:** लगातार check करता है call stack खाली है या नहीं

Flow:

text

1. `setTimeout` Web API में जाता है
2. 5 seconds complete होने पर Task Queue में add होता है
3. Event Loop check करता है Call Stack खाली है क्या?
4. अगर खाली है → Task Queue से pop करके Call Stack में डालता है
5. Call Stack में execute होता है
6. Complete होने पर हट जाता है

Key Points:

- Call Stack सबसे पहले चलता है
- Web APIs parallel में चलते हैं

- Task Queue में complete हुए tasks wait करते हैं
- Event Loop bridge का काम करता है

Section 5: Performance Measurement

Performance क्या होता है?

- Website कितनी fast load होती है
- User को कितना wait करना पड़ता है

Browser Performance Inspector

- DevTools → Performance Tab
- Green color = अच्छा performance
- **LCP (Largest Contentful Paint)**: सबसे बड़ा element कब load हुआ
- **First Paint**: पहला content कब दिखा
- **Reflow**: Layout change
- **Repaint**: Color/Style change

Local vs Real World

- **Local**: 0.06 seconds (network नहीं)
- **Real World**: 2-5 seconds (Network Involved)
- **Good Score**: 51 seconds से कम LCP

Quick Reference Card

```
js

// Module Pattern
const Module = (function() {
  let private = 10;
  function publicFn() { return private; }
  return { publicFn };
})();

// Factory Function
function createObject(name) {
  return {
    name,
    method() { }
  };
}

// Observer Pattern
class Subject {
  constructor() { this.observers = []; }
  subscribe(obs) { this.observers.push(obs); }
  notify(msg) { this.observers.forEach(o => o.update(msg)); }
}
```

```

// Debounce
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

// Throttle
function throttle(fn, delay) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= delay) {
      last = now;
      fn(...args);
    }
  };
}

// Lazy Loading
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.src = entry.target.dataset.src;
      observer.unobserve(entry.target);
    }
  });
});

// Code Splitting
const { fn } = await import("./module.js");

// Document Fragment
const frag = document.createDocumentFragment();
// ... append elements
parent.appendChild(frag);

// Call Stack Order
// LIFO - Last In First Out

// Event Loop
// Stack → Web API → Task Queue → Event Loop → Stack

```

Important Takeaways

1. **Design Patterns** = College vs Real Engineer का फर्क
2. **Debounce** = Actions के बीच gap
3. **Throttle** = Fixed interval
4. **Lazy Loading** = View में आने पर load
5. **Document Fragment** = Memory में batch, फिर DOM में add
6. **Memory Leaks** = Timers/Listeners clear करना जरूरी
7. **Code Splitting** = जरूरत पर load

8. **Separation of Concerns** = Logic अलग, DOM अलग
9. **Call Stack** = Function execution tracking (LIFO)
10. **Web APIs** = Browser के features
11. **Event Loop** = Stack खाली होने पर Queue से उठाओ

🎯 Final Tips:

- **Code रटा नहीं जाता** - Approach समझो
- **ChatGPT/Google use करना कोई शर्म नहीं**
- **Consistency** = Success
- जो समझ ना आए - **3-4 बार देखो**
- **Project बनाओ** - Practical से ही आएगा

make full complete notes from part 1 to part 4 in english with in depth explanation and proper examples

JavaScript Complete Master Notes (Part 1 - 4)

TABLE OF CONTENTS

1. JavaScript Fundamentals
2. DOM & Browser APIs
3. Asynchronous JavaScript
4. Advanced Topics & Architecture
5. Quick Reference

PART 1: JavaScript Fundamentals

1. JavaScript History & Background

Origin Story

- **Brendan Eich** created JavaScript in **1995** at **Netscape**
- Built in only **15 days**
- Original name: **Mocha** → **LiveScript** → **JavaScript**
- Named "JavaScript" for marketing (Java was popular then)
- **JavaScript has NO relation to Java** — completely different languages

ECMAScript

- **ECMA** (European Computer Manufacturers Association) maintains the standard
- **ES5** (2009) → **ES6/ES2015** (major update) → ES2016, ES2017...

- New features added yearly

2. Variables & Declarations

The Three Keywords

Keyword	Scope	Re-declare	Re-assign	Hoistin
var	Function	✓ Yes	✓ Yes	undefi
let	Block	✗ No	✓ Yes	TDZ Er
const	Block	✗ No	✗ No	TDZ Er

Declaration vs Initialization

```
js

// Declaration - variable created, no value
let a;

// Initialization - first value assigned
a = 12;

// Declaration + Initialization together
let b = 12;

// Re-assignment - updating value
b = 32;

// Re-declaration (only with var)
var c = 10;
var c = 20; // ✓ allowed with var
```

Hoisting

Hoisting = JavaScript moves declarations to top of their scope before execution

```
js

// var - hoisted with undefined value
console.log(x); // undefined (not error!)
var x = 12;

// Let/const - hoisted but in TDZ
console.log(y); // ✗ ReferenceError: Cannot access 'y' before initialization
let y = 12;
```

Temporal Dead Zone (TDZ)

The time between **declaration** and **initialization** where variable can't be accessed.

```
js
```

```
// TDZ starts here for `a`
console.log(a); // ❌ Error: Cannot access 'a' before initialization
let a = 12;      // TDZ ends here
console.log(a); // ✅ 12
```

Visual:

text

```
Line 1: ---- TDZ STARTS ----
Line 2: console.log(a) ❌ ERROR
Line 3: let a = 12      ✅ TDZ ENDS
Line 4: console.log(a) ✅ Works
```

var vs let vs const - Complete Comparison

js

```
// var - Function scoped, avoid using
function test() {
  var x = 10;
  if (true) {
    var x = 20; // Same variable!
    console.log(x); // 20
  }
  console.log(x); // 20
}

// Let - Block scoped, use for changing values
function test2() {
  let y = 10;
  if (true) {
    let y = 20; // Different variable
    console.log(y); // 20
  }
  console.log(y); // 10
}

// const - Block scoped, can't reassign
const PI = 3.14;
PI = 3.15; // ❌ TypeError: Assignment to constant variable
```

const with Objects (Important!)

js

```
const person = { name: "Harsh" };

// ✅ Can modify properties
person.name = "Harshita";
person.age = 26;

// ❌ Can't reassign the entire object
person = { name: "New" }; // TypeError
```

Why? const prevents re-assignment, not mutation.

3. Scope

Types of Scope

1. Global Scope - Accessible everywhere

```
js

let globalVar = "I'm everywhere";

function test() {
  console.log(globalVar); //  Works
}
console.log(globalVar); //  Works
```

2. Function Scope - Only inside function

```
js

function test() {
  let funcVar = "I'm local";
  console.log(funcVar); //  Works
}
console.log(funcVar); //  ReferenceError
```

3. Block Scope - Only inside {}

```
js

if (true) {
  let blockVar = "I'm block scoped";
  console.log(blockVar); //  Works
}
console.log(blockVar); //  ReferenceError
```

Lexical Scoping (Physical Position)

JavaScript uses **lexical (physical) scoping** - the position where you **write** the code determines access.

```
js

let a = 12;

function outer() {
  let b = 13;

  function inner() {
    let c = 14;
    // Can access a, b, c
    console.log(a, b, c); // 12, 13, 14
  }

  inner();
  // Can access a, b - NOT c
}
```

Nested Function Scope Example

```
js

function A() {
  let a = 1;

  function B() {
    let b = 2;

    function C() {
      let c = 3;
      console.log(a, b, c); // ✅ All accessible
    }

    C();
    // console.log(c); ❌ Not accessible
  }

  B();
  // console.log(b); ❌ Not accessible
}
```

4. Data Types

Primitive Types (7 types)

```
js

// 1. String
let name = "Harsh";
let single = 'Single quotes';
let backtick = `Backticks allow ${name}`; // Template Literals

// 2. Number (integers and floats both)
let age = 26;
let price = 99.99;

// 3. Boolean
let isLoggedIn = true;
let isAdmin = false;

// 4. Null - Intentional absence of value
let selectedUser = null; // We'll fill this later

// 5. Undefined - Value not assigned
let notSet; // undefined by default

// 6. Symbol - Unique identifier
const id = Symbol("id");

// 7. BigInt - For very large numbers
const bigNumber = 9007199254740993n;
```

Reference Types (3 types)

```
js

// 1. Array
let arr = [1, 2, 3];

// 2. Object
let obj = { name: "Harsh", age: 26 };

// 3. Function
let fn = function() { };
```

Copy Behavior: Primitive vs Reference

Primitives - Real Copy:

```
js

let a = 12;
let b = a; // Real copy
b = 15;

console.log(a); // 12 (unchanged)
console.log(b); // 15
```

References - Reference Copy:

```
js

let arr1 = [1, 2, 3];
let arr2 = arr1; // Reference copy, NOT real copy
arr2.push(4);

console.log(arr1); // [1, 2, 3, 4] ❌ Also changed!
console.log(arr2); // [1, 2, 3, 4]
```

Why? Both point to same memory location.

To make real copy:

```
js

// Shallow copy
let arr2 = [...arr1];
let obj2 = { ...obj1 };

// Deep copy (for nested)
let deep = JSON.parse(JSON.stringify(obj1));
```

null vs undefined

null	undefined
You assign it intentionally	JavaScript assigns it
"Value is absent on purpose"	"Value not yet provided"
let x = null;	let x;

Real example:

```
js

// undefined - User hasn't typed yet
let searchQuery; // undefined

// null - User selected "no preference"
let selectedTheme = null; // intentionally empty
```

typeof Quirks

```
js

typeof "hello"    // "string"
typeof 42         // "number"
typeof true       // "boolean"
typeof undefined  // "undefined"
typeof null       // "object" ⚠️ (Famous JavaScript bug!)
typeof []         // "object"
typeof {}         // "object"
typeof function(){} // "function"
typeof NaN        // "number" ⚠️ (NaN is a number?)
typeof 123n       // "bigint"
```

Truthy vs Falsy Values**Falsy (only 8 values):**

```
js

false
0
-0
0n      // BigInt zero
""      // Empty string
null
undefined
NaN
```

Everything else is Truthy:

```
js

!!0      // false
!!""     // false
!!null   // false
!!undefined // false
!!NaN    // false

!!1      // true
!!"hello" // true
!![]     // true (empty array is truthy!)
!!{}     // true (empty object is truthy!)
!!-1     // true (negative is truthy!)
!!"false" // true (string "false" is truthy!)
```

Trick to check: Double negation !!

js

```
!!"hello" // true
!!0       // false
```

5. Operators

Arithmetic Operators

js

```
console.log(10 + 5); // 15 (Addition)
console.log(10 - 5); // 5  (Subtraction)
console.log(10 * 5); // 50 (Multiplication)
console.log(10 / 5); // 2  (Division)
console.log(10 % 3); // 1  (Modulus - remainder)
console.log(2 ** 3); // 8  (Exponentiation - 2³)
```

Comparison Operators

js

```
// == (Loose equality) - Only value, NOT type
console.log(12 == "12"); // true ⚠️
console.log(0 == false); // true ⚠️
console.log(null == undefined); // true ⚠️

// === (Strict equality) - Value AND type ✅ USE THIS
console.log(12 === "12"); // false ✅
console.log(12 === 12);   // true  ✅

// != vs !==
console.log(12 != "12"); // false (Loose)
console.log(12 !== "12"); // true  (strict) ✅

// > < >= <=
console.log(10 > 5); // true
console.log(10 < 5); // false
console.log(10 >= 10); // true
console.log(10 <= 10); // true
```

Rule: Always use === and !== (strict)

Logical Operators

js

```
// && (AND) - Both must be true
console.log(true && true); // true
console.log(true && false); // false
console.log(10 > 5 && 20 > 10); // true

// || (OR) - Any one true
console.log(true || false); // true
console.log(false || false); // false
```

```

console.log(10 > 5 || 20 > 30); // true

// ! (NOT) - Inverts
console.log(!true); // false
console.log(!false); // true
console.log(!0); // true

```

Assignment Operators

```

js

let a = 10;

a += 5; // a = a + 5 → 15
a -= 3; // a = a - 3 → 12
a *= 2; // a = a * 2 → 24
a /= 4; // a = a / 4 → 6
a %= 4; // a = a % 4 → 2

```

Unary Operators

```

js

// Increment (++a vs a++)
let a = 5;
console.log(++a); // 6 (Pre-increment: increase THEN print)
console.log(a++); // 6 (Post-increment: print THEN increase)
console.log(a); // 7

// Decrement
let b = 5;
console.log(--b); // 4
console.log(b--); // 4
console.log(b); // 3

// Unary plus - converts to number
console.log(+ "5"); // 5 (number)
console.log(+ "hello"); // NaN

// Unary minus - negation
console.log(-5); // -5
console.log(- "5"); // -5 (number)

// typeof
console.log(typeof "hello"); // "string"
console.log(typeof 42); // "number"

// ! (NOT) - for truthy/falsy check
console.log(! "hello"); // false (because "hello" is truthy)

```

Ternary Operator

```

js

// Syntax: condition ? valueIfTrue : valueIfFalse

let age = 20;
let status = age >= 18 ? "Adult" : "Minor";

```

```

console.log(status); // "Adult"

// Chained ternary (like if-else if-else)
let score = 85;
let grade = score >= 90 ? "A" :
             score >= 80 ? "B" :
             score >= 70 ? "C" : "F";
console.log(grade); // "B"

```

Type Coercion (Important!)

```

js

// + with String → Concatenation
console.log("5" + 1);    // "51" (string)
console.log("5" + true); // "5true"
console.log(1 + 2 + "3"); // "33" (left to right: 1+2=3, then "3"+"3"="33")
console.log("1" + 2 + 3); // "123"

// - * / with String → Number conversion
console.log("5" - 1);    // 4
console.log("5" * 2);    // 10
console.log("10" / 2);   // 5

// Because `+` has TWO jobs:
// 1. Addition (for numbers)
// 2. Concatenation (for strings)
// Other operators only do math → force number conversion

```

Why + prefers string:

If ANY operand is a string, JavaScript assumes you want concatenation.

6. Control Flow

if / else if / else

```

js

let age = 18;

if (age >= 18) {
  console.log("Adult");
} else if (age >= 13) {
  console.log("Teenager");
} else {
  console.log("Child");
}

```

switch case

```

js

let day = "Monday";

switch (day) {

```

```

case "Monday":
  console.log("Start of week");
  break;
case "Friday":
  console.log("Almost weekend");
  break;
case "Saturday":
case "Sunday":
  console.log("Weekend!");
  break;
default:
  console.log("Regular day");
}

```

Always use `break` to avoid fall-through!

Early Return Pattern

Instead of nested if-else, return early:

```

js

// ❌ Nested
function getGrade(score) {
  if (score >= 90) {
    return "A";
  } else {
    if (score >= 80) {
      return "B";
    } else {
      if (score >= 70) {
        return "C";
      } else {
        return "F";
      }
    }
  }
}

// ✅ Early Return
function getGrade(score) {
  if (score >= 90) return "A";
  if (score >= 80) return "B";
  if (score >= 70) return "C";
  return "F";
}

```

7. Loops

For Loop

```

js

// Syntax: for (start; end; change)
for (let i = 1; i <= 5; i++) {
  console.log(i); // 1, 2, 3, 4, 5
}

```

```
// Reverse Loop
for (let i = 10; i >= 1; i--) {
  console.log(i); // 10, 9, 8, ..., 1
}

// Step by 2
for (let i = 0; i <= 10; i += 2) {
  console.log(i); // 0, 2, 4, 6, 8, 10
}
```

Use when: Start, End, and Change are all known.

While Loop

```
js

// Syntax: while (condition)
let i = 1;
while (i <= 5) {
  console.log(i);
  i++;
}
```

Use when: Only stopping condition is known.

Do-While Loop

```
js

// Runs at Least once
let i = 100;
do {
  console.log(i); // Prints 100 even though condition is false
  i++;
} while (i <= 5);
```

For...in (Object keys)

```
js

const user = { name: "Harsh", age: 26, city: "Bhopal" };

for (let key in user) {
  console.log(key, user[key]);
}

// name Harsh
// age 26
// city Bhopal
```

For...of (Array/Iterable values)

```
js

const fruits = ["apple", "banana", "mango"];

for (let fruit of fruits) {
```

```
    console.log(fruit);
  }
  // apple, banana, mango
```

Break & Continue

```
js

// break - stop the loop
for (let i = 1; i <= 10; i++) {
  if (i === 5) break;
  console.log(i); // 1, 2, 3, 4
}

// continue - skip current iteration
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue;
  console.log(i); // 1, 2, 4, 5
}
```

8. Functions

Function Declaration (Hoisted)

```
js

greet(); // ✅ Works (hoisted)

function greet() {
  console.log("Hello!");
}
```

Function Expression (Not Hoisted)

```
js

greet(); // ❌ Error: greet is not a function

const greet = function() {
  console.log("Hello!");
};
```

Arrow Function

```
js

// Syntax: () => {}
const greet = () => console.log("Hello");

// With parameters
const add = (a, b) => a + b;

// Multi-Line
const add = (a, b) => {
```

```
const result = a + b;
return result;
};
```

Key Differences:

Feature	Function	Arrow
this	Dynamic	From parent
Hoisting	Yes (declaration)	No
arguments object	Yes	No
Can be constructor	Yes	No

Parameters vs Arguments

```
js

// Parameters = variables in function definition
function add(a, b) { // a, b are parameters
  return a + b;
}

// Arguments = values passed when calling
add(5, 10); // 5, 10 are arguments
```

Default Parameters

```
js

function greet(name = "Guest") {
  console.log(`Hello, ${name}!`);
}

greet();           // Hello, Guest!
greet("Harsh");    // Hello, Harsh!
```

Rest Parameters (...)

```
js

function sum(...numbers) {
  return numbers.reduce((a, b) => a + b, 0);
}

console.log(sum(1, 2, 3, 4, 5)); // 15
console.log(sum(10, 20));        // 30
```

Spread Operator (...)

```
js
```

```
// Array spread
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1, 2, 3, 4, 5]

// Object spread
const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 }; // { a: 1, b: 2, c: 3 }

// Function arguments
const nums = [1, 2, 3];
console.log(Math.max(...nums)); // 3
```

Return Values

```
js

function add(a, b) {
  return a + b; // Returns value
}

function log(msg) {
  console.log(msg); // Returns undefined (no return)
}

// After return, nothing executes
function test() {
  return "Hello";
  console.log("Never runs"); // Unreachable
}
```

First-Class Functions

Functions are treated as values:

```
js

// 1. Store in variable
const fn = function() { };

// 2. Pass as argument
function caller(fn) {
  fn();
}
caller(() => console.log("Called"));

// 3. Return from function
function creator() {
  return function() {
    console.log("Created");
  };
}
```

Higher-Order Functions

Function that takes/returns another function:

```
js
```

```
// Takes function
function map(arr, fn) {
  const result = [];
  for (let item of arr) {
    result.push(fn(item));
  }
  return result;
}

// Returns function
function multiplier(x) {
  return function(y) {
    return x * y;
  };
}
const double = multiplier(2);
console.log(double(5)); // 10
```

Pure vs Impure Functions

Pure Function:

- Same input → Same output
- No side effects (doesn't modify outside)

```
js

// Pure
function add(a, b) {
  return a + b; // No side effects
}

// Impure
let total = 0;
function addToTotal(x) {
  total += x; // Modifies external state
  return total;
}
```

Closures

Function that remembers its parent's variables:

```
js

function outer() {
  let count = 0;

  return function inner() {
    count++;
    console.log(count);
  };
}

const counter = outer();
counter(); // 1
counter(); // 2
```

```
counter(); // 3
// count survives even after outer() finished!
```

How it works:

- When function returns another function
- The inner function keeps reference to parent's variables
- JavaScript creates a `[[Environment]]` backlink
- Variables don't get garbage collected

Private Counter Example:

```
js

function createCounter() {
  let count = 0; // Private!

  return {
    increment: () => ++count,
    decrement: () => --count,
    getCount: () => count
  };
}

const counter = createCounter();
counter.increment(); // 1
counter.increment(); // 2
counter.decrement(); // 1
// counter.count ❌ Not accessible - truly private!
```

Lexical Scoping in Closures

```
js

function A() {
  let a = "A's variable";

  function B() {
    // Can access a (Lexically in A)
    console.log(a);
  }

  return B;
}

const fn = A();
fn(); // "A's variable"
```

IIFE (Immediately Invoked Function Expression)

```
js

// Syntax: Wrap in parentheses and call immediately
(function() {
  console.log("Runs immediately!");
})();
```

```
// With arrow function
(() => {
  console.log("Also runs immediately");
})();

// Purpose: Create private scope
const counter = (function() {
  let count = 0;
  return {
    increment: () => ++count
  };
})();
```

PART 2: DOM & Browser APIs

1. DOM (Document Object Model)

What is DOM?

- **DOM = Document Object Model**
- Full HTML page is a **Tree Structure**
- Every element is a **Node**
- **DOM Manipulation** = Changing the webpage dynamically

Tree Structure

text

document

```
└─ html
   └─ head
      │ └─ title
      │ └─ meta
   └─ body
      │ └─ h1
      │ │ └─ "Hello"
      │ └─ div
      │ │ └─ p
      │ │ └─ span
      └─ script
```

Node Types

1. **Element Node** - HTML tags (<div> , <h1>)
2. **Text Node** - Text inside elements
3. **Comment Node** - HTML comments

Real-World Examples

- **Dropdown Menu** - Opens on click
- **Close Button** - Removes banner

- **Dark Mode Toggle** - Changes theme
- **Image Carousel** - Slides images
- **Toast Notifications** - Appears and disappears

2. Element Selection

Modern Methods (Recommended)

```
js

// querySelector - Returns FIRST matching element
document.querySelector('h1');
document.querySelector('.my-class');
document.querySelector('#my-id');
document.querySelector('div > p');

// querySelectorAll - Returns ALL matching (NodeList)
document.querySelectorAll('.card');
document.querySelectorAll('li');
```

Legacy Methods (Still Used)

```
js

// getElementById - Fast, only IDs
document.getElementById('my-id'); // No # needed

// getElementsByClassName - Returns HTMLCollection
document.getElementsByClassName('my-class');

// getElementsByTagName
document.getElementsByTagName('div');
```

Comparison Table

Method	Returns	Type	Live?
querySelector	First match	Element	No
querySelectorAll	All matches	NodeList	No
getElementById	Element	Element	No
getElementsByClassName	All matches	HTMLCollection	Yes

Pro Tip: Use `console.dir()` to see all properties of an element.

```
js

const h1 = document.querySelector("h1");
console.dir(h1); // Shows all properties
console.log(h1); // Sometimes shows just `<h1>`
```

3. Text/HTML Manipulation

```
js

const el = document.querySelector("h1");

// innerText - Visible text, respects CSS
el.innerText = "Hello";

// textContent - ALL text including hidden, FASTER ✔
el.textContent = "Hello";

// innerHTML - Parses HTML tags
el.innerHTML = "<i>Hello</i>"; // Renders italic
```

Difference Example

```
html

<h1>Hello <span style="display:none">Hidden</span> World</h1>

js

el.innerText // "Hello World" (hidden not shown)
el.textContent // "Hello Hidden World" (everything)
el.innerHTML // "Hello <span style='display:none'>Hidden</span> World"
```

When to Use

Property	Use When
textContent	Only text, speed matters ✔
innerText	Text as user sees it
innerHTML	Need to inject HTML

4. Attribute Manipulation

```
js

const link = document.querySelector("a");

// Get attribute
link.getAttribute("href"); // Returns value

// Set attribute
link.setAttribute("href", "https://google.com");
link.setAttribute("target", "_blank");

// Remove attribute
link.removeAttribute("target");
```

```
// Direct property access (sometimes works)
link.href = "https://google.com"; // Simpler
```

Common Attributes

```
js

img.src = "image.jpg";
img.alt = "Description";
input.value = "text";
input.placeholder = "Enter text";
input.disabled = true;
div.title = "Tooltip text";
```

5. Style Manipulation

Direct Style (Simple changes)

```
js

const el = document.querySelector("h1");

el.style.color = "red";
el.style.backgroundColor = "yellow";
el.style.fontSize = "24px";
el.style.display = "none";
```

Class-Based (Recommended)

```
js

// Add/remove classes
el.classList.add("active");
el.classList.remove("hidden");
el.classList.toggle("dark"); // Add if missing, remove if present
el.classList.contains("active"); // Returns true/false
el.classList.replace("old", "new");
```

Why Class-Based is Better:

- Multiple properties at once
- Cleaner code
- Easier to maintain
- Better performance

6. Dynamic DOM Manipulation

Creating Elements

```
js
```

```
const div = document.createElement("div");
div.textContent = "Hello";
div.className = "card";
div.classList.add("active");
```

Adding Elements

```
js

// append - adds to END
parent.append(child);

// prepend - adds to BEGINNING
parent.prepend(child);

// insertBefore - specific position
parent.insertBefore(newElement, referenceElement);

// appendChild - similar to append (older)
parent.appendChild(child);
```

Removing Elements

```
js

// Remove self
element.remove();

// Remove from parent
parent.removeChild(child);
```

Replacing Elements

```
js

oldElement.replaceWith(newElement);
```

Complete Example

```
js

// Create
const li = document.createElement("li");
li.textContent = "New Task";
li.className = "task-item";

// Add
const ul = document.querySelector("ul");
ul.appendChild(li);

// Remove after 3 seconds
setTimeout(() => li.remove(), 3000);
```

7. Events & Event Handling

What is an Event?

- Any action that happens on the page
- Click, hover, type, scroll, submit, etc.
- **Event** = Action
- **Listener** = Code that responds to action

addEventListener

```
js

element.addEventListener("eventName", function(event) {
    // Code runs when event happens
});
```

Common Events

Event	When it fires
click	Element clicked
dblclick	Double click
mouseover	Mouse enters element
mouseout	Mouse leaves element
mousemove	Mouse moves
keydown	Key pressed
keyup	Key released
input	Input value changes
change	Select/checkbox changed
submit	Form submitted
focus / blur	Get/lose focus
load	Page loaded
scroll	Page scrolled

removeEventListener

```
js

function handler() {
    console.log("Clicked");
}
```

```
btn.addEventListener("click", handler);
btn.removeEventListener("click", handler); // Must use same function
```

Event Object

```
js

element.addEventListener("click", (e) => {
  console.log(e.target);    // Element that fired event
  console.log(e.type);      // "click"
  console.log(e.clientX);   // Mouse X coordinate
  console.log(e.clientY);   // Mouse Y coordinate
  console.log(e.key);       // For keyboard events

  e.preventDefault(); // Stops default behavior
});
```

preventDefault()

```
js

// Stop form from reloading
form.addEventListener("submit", (e) => {
  e.preventDefault();
  // Do custom validation
});

// Stop link navigation
link.addEventListener("click", (e) => {
  e.preventDefault();
  // Do something else
});
```

8. Event Propagation

Event Bubbling (Default)

Event fires on the target element **FIRST**, then **bubbles up** to parents.

```
html

<div id="parent">
  <div id="child">
    <button id="btn">Click</button>
  </div>
</div>

js

btn.addEventListener("click", () => console.log("Button"));
child.addEventListener("click", () => console.log("Child"));
parent.addEventListener("click", () => console.log("Parent"));

// Click button → Output:
```

```
// Button
// Child
// Parent
```

Event Capturing

Event goes from **top to bottom** first, then bubbles.

```
js

// Pass `true` as third argument
element.addEventListener("click", handler, true);
```

Event Flow Phases

text

1. Capturing Phase (Top → Down)
document → html → body → parent → child
2. Target Phase
child (target)
3. Bubbling Phase (Bottom → Up)
child → parent → body → html → document

stopPropagation()

```
js

btn.addEventListener("click", (e) => {
  e.stopPropagation(); // Stops bubbling
  console.log("Only this runs");
});
```

Event Delegation

Attach **ONE** listener to parent instead of many to children.

```
js

// ❌ Bad - 5 Listeners
document.querySelectorAll("li").forEach(li => {
  li.addEventListener("click", () => li.remove());
});

// ✅ Good - 1 Listener on parent
const ul = document.querySelector("ul");
ul.addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    e.target.remove();
  }
});
```

Benefits:

- Fewer listeners (performance)
- Works for dynamically added elements
- Cleaner code

9. Forms & Validation

Reading Input Values

```
js

const input = document.querySelector("#email");

console.log(input.value);      // Input's value
console.log(div.textContent);  // Div's text content
// ❌ input.textContent → undefined
// ❌ div.value → undefined
```

Rule: `.value` for inputs, `.textContent` for text elements.

Form Submission

```
js

form.addEventListener("submit", (e) => {
  e.preventDefault(); // Stop page reload

  const email = emailInput.value;
  const password = passwordInput.value;

  // Validate
  if (email === "") {
    console.log("Email required");
    return;
  }

  // Send to server
  console.log("Submitting:", { email, password });
});
```

HTML5 Validation Attributes

```
html



```

Note: These can be bypassed by developers, so also use JS validation.

Regex Validation

```
js
```

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const isValid = emailRegex.test(email);

if (!isValid) {
  console.log("Invalid email");
}
```

Pro Tip: Don't write regex from scratch. Use ChatGPT/Google.

10. Local Storage

What is Local Storage?

- Browser's database
- Data persists across sessions
- Only stores **strings**
- Limit: ~5 MB

Methods

```
js

// Save
localStorage.setItem("username", "Harsh");

// Get
const user = localStorage.getItem("username");

// Remove one
localStorage.removeItem("username");

// Remove all
localStorage.clear();
```

Storing Objects/Arrays

```
js

// Store - Convert to string
const user = { name: "Harsh", age: 26 };
localStorage.setItem("user", JSON.stringify(user));

// Retrieve - Parse back
const stored = localStorage.getItem("user");
const userObj = JSON.parse(stored);
console.log(userObj.name); // "Harsh"
```

11. Session Storage

```
js
```

```
// Same API as LocalStorage
sessionStorage.setItem("temp", "data");
sessionStorage.getItem("temp");
sessionStorage.removeItem("temp");
sessionStorage.clear();
```

Difference: Session storage clears when **tab closes**.

Comparison

Feature	LocalStorage	SessionStorage	Cookies
Expiry	Never	Tab close	Manual
Size	~5 MB	~5 MB	~4 KB
Sent to server	No	No	Yes
Type	String	String	String

12. Cookies

js

```
// Set cookie
document.cookie = "username=Harsh";

// Set with expiry
document.cookie = "username=Harsh; expires=Fri, 31 Dec 2027 23:59:59 GMT";

// Read all (returns single string)
console.log(document.cookie);
```

Cookie Format:

text

"username=Harsh; theme=dark; lang=en"

PART 3: Asynchronous JavaScript

1. Synchronous vs Asynchronous

Synchronous (Sync)

Line by line execution. Each line waits for previous.

js

```
console.log("A");
console.log("B");
```

```
console.log("C");  
// Output: A, B, C
```

Asynchronous (Async)

Doesn't wait. Multiple things can run.

```
jsconsole.log("A");  
  
setTimeout(() => console.log("B"), 2000);  
console.log("C");  
// Output: A, C, B (B after 2 seconds)
```

Why Async?

```
js  
  
// ❌ Problematic synchronous  
const data = fetchDataFromServer(); // Blocks for 5 seconds!  
console.log("Page frozen for 5s");  
  
// ✅ Async  
fetchDataFromServer((data) => {  
  console.log(data); // Runs when ready  
});  
console.log("Page responsive!"); // Runs immediately
```

2. Callbacks

What is a Callback?

A function passed as an argument to another function.

```
js  
  
function processData(data, callback) {  
  // Do something with data  
  callback(data);  
}  
  
processData("Hello", (result) => {  
  console.log(result); // "Hello"  
});
```

Real Example

```
js  
  
function getUser(id, callback) {  
  setTimeout(() => {  
    callback({ id, name: "Harsh" });  
  }, 1000);  
}
```

```
getUser(1, (user) => {  
  console.log(user); // { id: 1, name: "Harsh" } after 1 second  
});
```

3. Callback Hell

The Problem

```
js  
  
getUser(1, (user) => {  
  getPosts(user.id, (posts) => {  
    getComments(posts[0].id, (comments) => {  
      getReplies(comments[0].id, (replies) => {  
        // Nested madness!  
        console.log(replies);  
      });  
    });  
  });  
});
```

This is called **Callback Hell** or **Pyramid of Doom**.

Problems:

- Hard to read
- Hard to debug
- Hard to maintain
- Error handling difficult

4. Promises

What is a Promise?

Represents a **future value** from an async operation.

States:

- **Pending** - Initial state
- **Fulfilled** - Operation completed (resolved)
- **Rejected** - Operation failed

Creating a Promise

```
js  
  
const promise = new Promise((resolve, reject) => {  
  // Do async work  
  setTimeout(() => {  
    const success = Math.random() > 0.5;  
    if (success) {
```

```
    resolve("Success data");
  } else {
    reject("Error occurred");
  }
}, 1000);
});
```

Using a Promise

js

```
promise
  .then((data) => {
    console.log("Success:", data);
  })
  .catch((error) => {
    console.log("Error:", error);
  })
  .finally(() => {
    console.log("Always runs");
  });
```

Promise Chaining

js

```
fetchUser(1)
  .then((user) => fetchPosts(user.id))
  .then((posts) => fetchComments(posts[0].id))
  .then((comments) => console.log(comments))
  .catch((error) => console.log("Error:", error));
```

Much cleaner than callback hell!

5. Async/Await

Syntax Sugar for Promises

js

```
async function fetchData() {
  try {
    const user = await fetchUser(1);
    const posts = await fetchPosts(user.id);
    const comments = await fetchComments(posts[0].id);
    console.log(comments);
  } catch (error) {
    console.log("Error:", error);
  }
}

fetchData();
```

Key Rules

1. `await` can only be used inside `async` function
2. `async` function always returns a `Promise`
3. Use `try/catch` for error handling

Async Function Returns Promise

```
js

async function test() {
  return 42;
}

// These are same:
test().then((val) => console.log(val)); // 42

// Or
const val = await test(); // 42
```

Parallel vs Sequential

```
js

// Sequential - slow
const user = await fetchUser();
const posts = await fetchPosts();
// Total: 2 + 3 = 5 seconds

// Parallel - fast 
const [user, posts] = await Promise.all([
  fetchUser(), // 2s
  fetchPosts() // 3s
]);
// Total: 3 seconds (whichever is slower)
```

6. Fetch API

Basic Fetch

```
js

fetch("https://api.example.com/users")
  .then(response => response.json()) // Parse JSON
  .then(data => console.log(data))
  .catch(error => console.log(error));
```

Why `response.json()` Returns Promise

- Response is a **Readable Stream**
- Not directly readable
- `.json()` converts stream to actual data

With Async/Await

```
js

async function getUser() {
  try {
    const response = await fetch("https://api.example.com/users/1");
    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.log(error);
  }
}
```

Fetch Options

```
js

fetch(url, {
  method: "POST", // GET, POST, PUT, DELETE
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer token"
  },
  body: JSON.stringify({
    name: "Harsh",
    age: 26
  })
});
```

Real-World Example

```
js

async function loadUsers() {
  const container = document.querySelector("#users");

  try {
    const res = await fetch("https://randomuser.me/api/?results=5");
    const data = await res.json();

    data.results.forEach(user => {
      const card = document.createElement("div");
      card.classList.add("user-card");
      card.innerHTML = `
        
        <h3>${user.name.first} ${user.name.last}</h3>
        <p>${user.email}</p>
      `;
      container.appendChild(card);
    });
  } catch (error) {
    container.innerHTML = "<p>Error loading users</p>";
  }
}
```

7. HTTP Status Codes

Code Range	Meaning
200-299	Success 
300-399	Redirect
400-499	Client Error (404 Not Found)
500-599	Server Error

Common Codes

- **200** - OK
- **201** - Created
- **400** - Bad Request
- **401** - Unauthorized
- **403** - Forbidden
- **404** - Not Found
- **500** - Internal Server Error

PART 4: Advanced Topics & Architecture

1. Design Patterns

Why Design Patterns?

- **Differentiate** junior dev from senior engineer
- **Reusable** solutions to common problems
- **Maintainable, Scalable, Modular** code
- For **lakhs of lines** of code
- Handle **public/private** separation

1.1 Module Pattern

Definition: Write code inside an **IIFE** to keep variables private.

```
js

const Bank = (function() {
  let balance = 12000; // Private

  function checkBalance() {
    console.log("Balance:", balance);
  }

  function withdraw(amount) {
```

```

    if (amount <= balance) {
      balance -= amount;
      console.log("Withdrew:", amount, "| New Balance:", balance);
    } else {
      console.log("Insufficient funds");
    }
  }
}

// Return only what should be PUBLIC
return {
  checkBalance,
  withdraw
};
})();

Bank.checkBalance(); // Balance: 12000
Bank.withdraw(11000); // Withdrew: 11000 | New Balance: 1000
// Bank.balance      // ❌ undefined - private!

```

Key Points:

- IIFE creates private scope
- Return object becomes public API
- Variables not returned are private

1.2 Revealing Module Pattern

Same as Module Pattern, but you can rename exports.

```

js

const Bank = (function() {
  let balance = 12000;

  function checkBalance() { console.log(balance); }
  function setBalance(val) { balance = val; }
  function withdraw(amt) { balance -= amt; }

  // Rename on return
  return {
    check: checkBalance, // renamed
    set: setBalance,      // renamed
    draw: withdraw        // renamed
  };
})();

Bank.check(); // 12000
Bank.set(5000);
Bank.draw(1000); // 4000

```

1.3 Factory Function Pattern

Definition: Function that creates and returns objects without `new` or classes.

```
js
```

```

function createProduct(name, price) {
  let stock = 10; // Private per object

  return {
    name,
    price,

    buy(quantity) {
      if (quantity <= stock) {
        stock -= quantity;
        console.log(`${quantity} booked | ${stock} left`);
      } else {
        console.error(`Out of stock! Only ${stock} available`);
      }
    },

    refill(quantity) {
      stock += quantity;
      console.log(`Refilled! Now ${stock} available`);
    },

    checkStock() {
      console.log(`Current stock: ${stock}`);
    }
  };
}

const iPhone = createProduct("iPhone", 70000);
const KitKat = createProduct("KitKat", 10);

iPhone.buy(6);           // 6 booked | 4 Left
KitKat.buy(5);           // 5 booked | 5 Left
iPhone.checkStock();    // Current stock: 4
KitKat.checkStock();    // Current stock: 5 (independent!)

```

Benefits:

- Each object has own memory
- No need for classes
- Private state via closures

1.4 Observer Pattern

Definition: When one object (Subject) changes, notify all dependents (Observers).

YouTube Subscribe Example:

```

js

class YouTubeChannel {
  constructor(name) {
    this.name = name;
    this.subscribers = [];
  }

  subscribe(user) {
    this.subscribers.push(user);
    user.update(`You subscribed to ${this.name}`);
  }
}

```

```

    }

    unsubscribe(user) {
      this.subscribers = this.subscribers.filter(s => s !== user);
      user.update(`You unsubscribed from ${this.name}`);
    }

    notify(message) {
      this.subscribers.forEach(sub => {
        sub.update(`${sub.name}: ${message}`);
      });
    }
  }
}

class User {
  constructor(name) {
    this.name = name;
  }

  update(message) {
    console.log(message);
  }
}

// Usage
const sheryians = new YouTubeChannel("Sheryians");

const harsh = new User("Harsh");
const amit = new User("Amit");

sheryians.subscribe(harsh);
// "You subscribed to Sheryians"

sheryians.subscribe(amit);
// "You subscribed to Sheryians"

sheryians.notify("New video is live!");
// "Harsh: New video is Live!"
// "Amit: New video is Live!"

sheryians.unsubscribe(harsh);
// "You unsubscribed from Sheryians"

sheryians.notify("Another video!");
// "Amit: Another video!" (Harsh no longer notified)

```

Used in: React, Redux, Event Emitters, RxJS

2. Performance Optimization

2.1 Debouncing

Definition: Only fire after user has STOPPED for a specific time.

Use Case: Search inputs, Form validation

js

```
function debounce(fn, delay) {
  let timer;

  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}

const searchInput = document.querySelector("#search");

searchInput.addEventListener("input", debounce(function(e) {
  console.log("Searching for:", e.target.value);
  // API call here
}, 500)); // Wait 500ms after user stops typing
```

Visual:

text

Type: H → reset timer
 Type: E → reset timer
 Type: L → reset timer
 Type: L → reset timer
 Type: O → reset timer
 Stop typing → 500ms passes → SEARCH!

2.2 Throttling

Definition: Fire at a FIXED interval regardless of how often user triggers.

Use Case: Scroll, Mouse move, Window resize

```
js

function throttle(fn, delay) {
  let last = 0;

  return function(...args) {
    const now = Date.now();

    if (now - last >= delay) {
      last = now;
      fn.apply(this, args);
    }
  };
}

window.addEventListener("scroll", throttle(function() {
  console.log("Scroll position:", window.scrollY);
}, 1000)); // Fire max once per second
```

Visual:

text

Scrolling continuously...
 Fire at 0s, 1s, 2s, 3s...
 Stop scrolling → stops firing

2.3 Lazy Loading Images

Concept: Load images only when they enter the viewport.

html

```



```

css

```
.lazy {
  opacity: 0;
  transition: opacity 0.3s;
}
.lazy.loaded {
  opacity: 1;
}
```

js

```
const images = document.querySelectorAll(".lazy");

const observer = new IntersectionObserver((entries, obs) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      img.src = img.dataset.src;
      img.classList.add("loaded");
      obs.unobserve(img); // Stop observing once loaded
    }
  });
}, {
  root: null, // viewport
  threshold: 0.1 // 10% visible
});

images.forEach(img => observer.observe(img));
```

Benefits:

- Faster initial page load
- Saves bandwidth
- Better user experience

2.4 Avoiding Reflows & Repaints

The Problem

Every DOM change forces the browser to:

1. **Reflow** (recalculate layout)
2. **Repaint** (redraw pixels)

Expensive!

Bad Example

```
js

for (let i = 0; i < 10000; i++) {
  const li = document.createElement("li");
  li.textContent = i;
  ul.appendChild(li); // ❌ Reflow 10000 times!
}
```

Good Example (Document Fragment)

```
js

const fragment = document.createDocumentFragment();

for (let i = 0; i < 10000; i++) {
  const li = document.createElement("li");
  li.textContent = i;
  fragment.appendChild(li); // ✅ In memory
}

ul.appendChild(fragment); // ✅ ONE reflow
```

CSS Version

```
js

// ❌ Multiple reflows
el.style.color = "red";
el.style.fontSize = "20px";
el.style.margin = "10px";

// ✅ Add class - one reflow
el.classList.add("highlighted");
```

2.5 Memory Leaks

Timer Leaks

```
js

// ❌ Leak - interval never cleared
setInterval(() => {
  console.log("Tick");
}, 1000);
```

```

    }, 1000);

    // ✅ Clear it
    let count = 0;
    const interval = setInterval(() => {
        count++;
        if (count >= 10) {
            clearInterval(interval);
            console.log("Done");
        }
    }, 1000);

```

Event Listener Leaks

```

js

// ❌ Leak
element.addEventListener("click", handler);

// ✅ Remove when done
element.removeEventListener("click", handler);

```

2.6 Code Splitting

Concept: Load code only when needed.

HTML:

```

html

<script type="module" src="main.js"></script>

```

main.js:

```

js

document.querySelector("#loadHeavy").addEventListener("click", async () => {
    // Dynamic import - Loads only when clicked
    const { heavyFunction } = await import("./heavy.js");
    heavyFunction();
});

```

heavy.js:

```

js

export function heavyFunction() {
    console.log("Heavy feature loaded!");
    // Lots of code here
}

```

Benefits:

- Faster initial load
- On-demand loading

- Better performance

3. Separation of Concerns

Definition: Keep DOM code and Logic code separate.

Bad Code

```
js

btn.addEventListener("click", () => {
  const a = Math.random() * 100;
  const b = Math.random() * 100;
  const sum = a + b; // Logic mixed with DOM

  const li = document.createElement("li");
  li.textContent = sum;
  ul.appendChild(li);
});
```

Good Code

```
js

// Logic - Separate function
function calculateSum(a, b) {
  return a + b;
}

// DOM - Separate concern
btn.addEventListener("click", () => {
  const a = Math.floor(Math.random() * 100);
  const b = Math.floor(Math.random() * 100);
  const sum = calculateSum(a, b); // Uses Logic

  const li = document.createElement("li");
  li.textContent = sum;
  ul.appendChild(li);
});
```

Benefits:

- Easier to test logic
- Reusable functions
- Cleaner code
- Same concept in React

4. Custom Utilities

Custom Map Function

```
js
```

```
function myMap(arr, callback) {
  const result = [];

  for (let i = 0; i < arr.length; i++) {
    result.push(callback(arr[i], i, arr));
  }

  return result;
}

// Usage
const numbers = [1, 2, 3, 4];
const doubled = myMap(numbers, (val) => val * 2);
console.log(doubled); // [2, 4, 6, 8]
```

Custom Filter Function

```
js

function myFilter(arr, callback) {
  const result = [];

  for (let i = 0; i < arr.length; i++) {
    if (callback(arr[i], i, arr)) {
      result.push(arr[i]);
    }
  }

  return result;
}

const evens = myFilter([1, 2, 3, 4, 5], (val) => val % 2 === 0);
console.log(evens); // [2, 4]
```

5. How JS Works in Browser

5.1 Call Stack

JavaScript is Single-Threaded — one thing at a time.

Call Stack:

- Functions added when called
- Removed when done
- LIFO (Last In, First Out)

Example

```
js

function A() {
  console.log("A");
}
```

```
function B() {
  A();
  console.log("B");
}

function C() {
  B();
  console.log("C");
}

C();
// Output: A, B, C
```

Step by Step:

Step	Action	Stack
1	C() called	[C]
2	B() called	[C, B]
3	A() called	[C, B, A]
4	A logs "A"	[C, B, A]
5	A finishes	[C, B]
6	B logs "B"	[C, B]
7	B finishes	[C]
8	C logs "C"	[C]
9	C finishes	[]

Stack Overflow

```
js

function infinite() {
  infinite(); // Recursion without end
}

infinite(); // ❌ Stack Overflow
```

5.2 Web APIs

Browser provides extra features (Web APIs) beyond JavaScript.

- JavaScript Language: ~400 features
- Browser Web APIs: ~100 features
- Total: ~500 features

Web APIs Include

- setTimeout , setInterval

- `fetch` , `XMLHttpRequest`
- `alert` , `prompt` , `confirm`
- `localStorage` , `sessionStorage`
- `console`
- `document` , `window`

Proof

Open browser console:

```
js  
  
window // See all Web APIs
```

Note: Node.js doesn't have these Web APIs. It has its own APIs.

5.3 Event Loop

Bridge between Call Stack and Task Queue.

The Flow

```
text  
  
1. Code runs → goes to Call Stack  
2. Async operations → sent to Web APIs  
3. When async completes → moves to Task Queue  
4. Event Loop checks: Is Call Stack empty?  
5. If empty → moves task from Queue to Stack  
6. Task runs → removed from Stack  
7. Repeat
```

Example

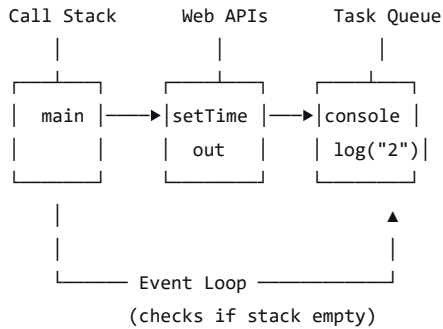
```
js  
  
console.log("1");  
  
setTimeout(() => {  
  console.log("2");  
}, 0);  
  
console.log("3");  
  
// Output: 1, 3, 2
```

Why?

1. `console.log("1")` → Stack → prints 1
2. `setTimeout` → Web API → waits 0ms → Task Queue
3. `console.log("3")` → Stack → prints 3
4. Stack empty → Event Loop moves `console.log("2")` from Queue → prints 2

Diagram

text



6. Performance Measurement

Chrome DevTools

- **Performance Tab** → Record
- **Green** = Good
- **Red** = Needs improvement

Key Metrics

Metric	Meaning
First Paint (FP)	First pixel painted
First Contentful Paint (FCP)	First content visible
Largest Contentful Paint (LCP)	Biggest element loaded
Time to Interactive (TTI)	Page becomes interactive

Local vs Real World

- **Local** (localhost): 0.06 seconds
- **Real World** (with network): 2-5 seconds

Good LCP: Under 2.5 seconds

7. Additional Important Topics

7.1 this Keyword

Context-dependent!

js

```
// 1. Global scope
console.log(this); // window (in browser)

// 2. Regular function
function test() {
  console.log(this); // window
}

// 3. Method (ES5 function)
const obj = {
  name: "Harsh",
  greet: function() {
    console.log(this); // obj (calling object)
  }
};
obj.greet();

// 4. Method (Arrow function)
const obj2 = {
  name: "Harsh",
  greet: () => {
    console.log(this); // window (from parent)
  }
};

// 5. Event Listener
btn.addEventListener("click", function() {
  console.log(this); // btn element
});

// 6. Class
class Person {
  constructor() {
    console.log(this); // new instance
  }
}
```

this Summary Table

Context	this Value
Global scope	window
Regular function	window
Method (ES5)	Calling object
Method (Arrow)	Parent's this
Event handler	Element with listener
Class	Instance object

7.2 call, apply, bind

Manual this binding.

```
js

function greet(greeting, punctuation) {
  console.log(`${greeting}, ${this.name}${punctuation}`);
}

const user = { name: "Harsh" };

// call - pass args individually
greet.call(user, "Hello", "!"); // "Hello, Harsh!"

// apply - pass args as array
greet.apply(user, ["Hi", "?"]); // "Hi, Harsh?"

// bind - returns new function
const boundGreet = greet.bind(user, "Hey");
boundGreet("..."); // "Hey, Harsh..."
```

Differences:

Method	Executes?	Args Style
call	✔ Immediately	Individual
apply	✔ Immediately	Array
bind	✘ Returns function	Individual

7.3 Classes

ES6 Syntax for Constructor Functions.

```
js

class User {
  constructor(name, email) {
    this.name = name;
    this.email = email;
    this.role = "user";
  }

  write(text) {
    console.log(`${this.name}: ${text}`);
  }

  checkRole() {
    return `You are a ${this.role}`;
  }
}

const harsh = new User("Harsh", "harsh@example.com");
harsh.write("Hello!"); // "Harsh: Hello!"
console.log(harsh.checkRole()); // "You are a user"
```

Inheritance

```
js

class User {
  constructor(name) {
    this.name = name;
    this.role = "user";
  }

  write(text) {
    console.log(`${this.name}: ${text}`);
  }
}

class Admin extends User {
  constructor(name) {
    super(name); // Call parent constructor
    this.role = "admin";
  }

  remove() {
    console.log("Removed!");
  }
}

const admin = new Admin("Harsh");
admin.write("Hi!"); // "Harsh: Hi!" (inherited)
admin.remove();    // "Removed!" (own method)
```

7.4 Prototypes

Shared memory for methods.

```
js

function CreatePencil(name, price, color) {
  this.name = name;
  this.price = price;
  this.color = color;
}

// Shared across ALL instances
CreatePencil.prototype.write = function(text) {
  const h1 = document.createElement("h1");
  h1.textContent = text;
  h1.style.color = this.color;
  document.body.appendChild(h1);
};

const p1 = new CreatePencil("Nataraj", 10, "black");
const p2 = new CreatePencil("Doms", 15, "red");

p1.write("Hello"); // Black text
p2.write("World"); // Red text
```

Benefits:

- Methods shared (memory efficient)
- All instances access prototype

- Can be modified at runtime

7.5 Prototypal Inheritance

Object inheriting from another object.

```
js

const coffee = {
  color: "dark",
  drink: function() {
    console.log("Good coffee!");
  }
};

const cappuccino = Object.create(coffee);
// cappuccino inherits from coffee

cappuccino.taste = "bitter";
cappuccino.drink(); // "Good coffee!"
console.log(cappuccino.color); // "dark"
```

Difference:

Classical	Prototypal
Class → Class	Object → Object
class X extends Y	Object.create(Y)
All OOP languages	JavaScript only

Quick Reference

Variable Keywords

```
js

var x = 1; // Function scope, avoid
let y = 2; // Block scope, reassign OK
const z = 3; // Block scope, no reassign
```

Data Types

```
js

// Primitives
"string", 42, true, null, undefined, Symbol(), 123n

// References
[], {}, function() {}
```

Operators

```
js

// Arithmetic
+ - * / % **

// Comparison (use ===)
== === != !== > < >= <=

// Logical
&& || !

// Assignment
= += -= *= /= %=

// Unary
++ -- + - ! typeof

// Ternary
condition ? true : false
```

Functions

```
js

// Declaration (hoisted)
function name() {}

// Expression (not hoisted)
const name = function() {};

// Arrow
const name = () => {};

// Parameters
function fn(a, b = 10) {}

// Rest
function fn(...args) {}

// Spread
const arr2 = [...arr1];
```

Arrays

```
js

arr.push(x);      // Add to end
arr.pop();        // Remove from end
arr.shift();      // Remove from start
arr.unshift(x);   // Add to start
arr.splice(i, n); // Remove n items from i
arr.slice(i, j);  // Copy from i to j
arr.map(fn);      // Transform
arr.filter(fn);   // Filter
arr.reduce(fn, init); // Reduce to single value
arr.find(fn);     // First match
```

```
arr.some(fn);      // Any match?  
arr.every(fn);     // All match?
```

Objects

```
js  
  
// Create  
const obj = { key: "value" };  
  
// Access  
obj.key;  
obj["key"];  
  
// Destructure  
const { name, age } = obj;  
  
// Copy (shallow)  
const copy = { ...obj };  
  
// Deep copy  
const deep = JSON.parse(JSON.stringify(obj));  
  
// Loop  
for (let key in obj) {}  
Object.keys(obj);  
Object.values(obj);  
Object.entries(obj);
```

DOM

```
js  
  
// Select  
document.querySelector("#id");  
document.querySelectorAll(".class");  
  
// Manipulate  
el.textContent = "text";  
el.innerHTML = "<b>html</b>";  
el.setAttribute("href", "url");  
el.classList.add("active");  
el.style.color = "red";  
  
// Create/Add/Remove  
const div = document.createElement("div");  
parent.append(div);  
parent.prepend(div);  
el.remove();  
  
// Events  
el.addEventListener("click", handler);  
el.removeEventListener("click", handler);
```

Async

```
js
```

```
// Promise
const p = new Promise((resolve, reject) => {
  resolve(42);
});
p.then(v => console.log(v)).catch(e => console.log(e));

// Async/Await
async function fn() {
  try {
    const val = await promise;
    console.log(val);
  } catch (err) {
    console.log(err);
  }
}

// Fetch
fetch(url)
  .then(res => res.json())
  .then(data => console.log(data));

// With await
const res = await fetch(url);
const data = await res.json();
```

Design Patterns

```
js

// Module Pattern
const Module = (function() {
  let private = 10;
  return { publicFn: () => private };
})();

// Factory Function
function createProduct(name) {
  return { name, method() {} };
}

// Observer
class Subject {
  constructor() { this.observers = []; }
  subscribe(o) { this.observers.push(o); }
  notify(msg) { this.observers.forEach(o => o.update(msg)); }
}
```

Performance

```
js

// Debounce
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}
```

```

// Throttle
function throttle(fn, delay) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= delay) {
      last = now;
      fn(...args);
    }
  };
}

// Lazy Load
const observer = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.src = entry.target.dataset.src;
    }
  });
});

// Document Fragment
const frag = document.createDocumentFragment();
// Add elements to frag
parent.appendChild(frag);

```

Binding

```

js

fn.call(obj, a, b);    // Immediate, args separate
fn.apply(obj, [a, b]); // Immediate, args array
const newFn = fn.bind(obj, a); // Returns new function
newFn(b);

```

Storage

```

js

// LocalStorage (persists)
localStorage.setItem("key", "value");
localStorage.getItem("key");
localStorage.removeItem("key");
localStorage.clear();

// SessionStorage (clears on tab close)
sessionStorage.setItem("key", "value");

// Cookies (small, sent to server)
document.cookie = "key=value; expires=date";

```

Final Words

Golden Rules

1. **Code is not memorized** - approach is understood
2. **Google/ChatGPT** are tools, use them
3. **Consistency** beats intensity
4. **Practice** by building projects
5. **Watch** tough topics 3-4 times
6. **Write notes** by hand - improves retention
7. **Read error messages** carefully

Learning Path

text

JavaScript Basics → DOM → Async → Advanced Patterns

↓

Build Projects (Toast, Notes, User Cards)

↓

React / Node.js (Next Steps)

Resources

- **MDN Web Docs** - Reference
- [JavaScript.info](#) - Deep Dives
- **ChatGPT** - Explanation of concepts
- **Stack Overflow** - Problem solving
- **YouTube** - Video tutorials

Remember:

"You can't memorize all of it. Nobody does. You just need to know how to approach problems and where to find answers."

Stay consistent. Keep building. You've got this! 🚀